Final Year Project Design

Kids Vaccination Booking



Final Year Design Project Proposal

By

Sehar Younas	**077007**
Hifza Shahzad	**077025**
Project ID	**BSIT22-G30**

Project Advisor:

Prof. Imran Shafiq

Bachelor of Science in IT (2022-2026)

Faculty of Computing & Information Technology (FCIT),

Govt. Islamia Graduate College, Gujranwala

Kids Vaccination Booking

A project presented to

University of The Punjab, Lahore

In Partial fulfilment of

requirement for the degree of

Bachelors of Science in Information Technology

Session (2022-2026)

By

Sehar Younas **077007**

Hifza Shahzad **077025**

Faculty of Computing & Information Technology (FCIT),

Govt. Islamia Graduate College, Gujranwala

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.ar

Sehar Younas [077007]

Signature: _____

Hifza Shahzad [077025]

Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2022-26), the project **Kids Vaccination Booking: An online vaccination platform for kids** was developed by **Sehar Younas (2022-icg-185), Hifza Shahzad (2021-gwg-354)** under the supervision of **Prof. Muhammad Imran Shafiqe.** in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology.**

FYP Advisor: Prof. Muhammad Imran Shafiqe Sb.

Advisor Signature: _____

Signatures (Faculty Advisory Committee (FAC))

	FAC1	**FAC2**
Name	Prof. Muhammad Imran	Prof. Muhammad Usman
Signature		

Head of FYP Coordination Office: Prof. Muhammad Imran Sb.

Signature: _____

Head Of Department, Information Technology: Prof. Muhammad Usman

Signature: _____ Dated: _____

Executive Summary

The Kids Vaccination Booking is a website that help in maintaining child vaccination courses. This platform is designed to give proper awareness to users about importance of vaccine. This website ensures that children should be vaccinated with a suitable vaccine based on their ages. The website ensures that users are directly connected with hospitals. It also provides the auto scheduling where users can get the vaccine schedule automatically based on their child ages. Users can also use the available slots of hospitals. The website enables users to get timely reminders through SMS/E-mail and saving the previous record. There is a separate admin panel where admin verify the users and hospitals approval, manage reviews and monitor overall system.

The website is developed by MERN stack in which (React.js for Frontend Node.js for Backend, MongoDB for data base and Express.js for connecting Backend a database) and many other technologies. The website gain trust of people by allowing them to book and take appointments in their desired hospitals. The developing website ensures a reliable system for users that develop awareness in people so that parent can prevent their child from fatal diseases, whose treatment is very expensive. Through this platform children can get complete vaccination doses and maintain their immunity. This design provides simplicity, reliability and security ensuring to provide a trustworthy and effective platform for connecting hospitals with users.

Acknowledgment

We express our gratitude to our Almighty Allah, and after that, we are thankful to us families, who allowed us to get higher education, and we became capable of creating something productive. We're also thankful to our project advisor and department faculty for their support.

Sehar Younas [077007]

Signature: _____

Hifza Shahzad [077025]

Signature: _____

Abbreviation

Table of all the abbreviations and acronyms used in your FY, P, along with their respective brief description. The abbreviation list must be ordered alphabetically.

Abbreviation	Description
API	Application programming interface Away for applications to communicate with each other.
DB	Database An organized collection of data.
DOB	Date of birth
FYDP	Final Year Design Project
GB	Gigabyte A unit of digital storage.
Info	information
Js	JavaScript Scripting language for web development
JSON	JavaScript Object Notation Lightweight format for storing and exchange data.
JWT	JSON Web Token Used for secure authentication
MERN	MongoDB, Express, React, Node Stack of Languages used to build web Applications.
npm	Node.js package manager Tool Used to Install, manage and share javaScript packages.
OTP	One time password Temporary code used for verification
PC	Personal computer
RAM	Random Access Memory Computer's short memory used to store temporary data while running computer.
RDBMS	Relational database management system System for managing data in related tables.
SMS	Short message service Text messaging service for sending short messages.
SMTP	Simple Mail Transfer Protocol Used for sending Emails.
UI	User interface Visual part of an application that user interact with.

Table of contents

Introduction 18

1.1 Problem statement 18

1.2 Problem solution 18

1.3 Objectives 18

1.4 Scope 19

1.5 System Components 19

1.5 Related System Analysis/ Literature Review 21

1.7 Vision statement 22

1.8 System Limitations and Constraints 22

1.9 Tools and Technologies 23

1.10 Work division 23

1.10 Project Deliverables 24

1.11 Project Planning 24

1.12 Summary 24

Requirement Analysis 25

2 Analysis 26

2.1 User classes and characteristics 26

2.2 Requirement Identifying Techniques 26

2.2.1 Interviews 26

2.2.2 Ethnography and Observation 26

2.2 Use Case Analysis 26

2.2.1 Use case 1 26

2.2.2 Use case 2 27

2.2.3 Use case 3 28

2.2.4 Use case 4 28

2.2.5 Use case 5 29

2.2.6 Use case 6 30

2.2.7 Use case 7 30

2.2.8 Use case 8 31

2.2.9 Use case 9 31

2.2.10 Use case 10 32

2.2.11 Use case 11 32

2.2.12 Use case 12 33
2.2.13 Use case 13 33
2.2.14 Use case 14 34
2.2.15 Use case 15 35
2.2.16 Use case 16 35
2.2.17 Use case Diagram 36
2.4 Functional Requirements 37
2.4.1 Functional Requirements 1 37
2.4.2 Functional Requirements 2 37
2.4.3 Functional Requirements 3 38
2.3.4 Functional Requirement 4 38
2.3.5 Functional Requirement 5 38
2.3.6 Functional Requirement 6 39
2.3.7 Functional Requirement 7 39
2.3.8 Functional Requirement 8 40
2.3.9 Functional Requirement 9 40
2.3.10 Functional Requirement 10 40
2.3.11 Functional Requirement 11 41
2.3.12 Functional Requirement 12 41
2.3.13 Functional Requirement 13 41
2.3.14 Functional Requirement 14 42
2.3.15 Functional Requirement 15 42
2.3.16 Functional Requirement 16 43
2.3.17 Functional Requirement 17 43
2.4 Non-Functional Requirements 44
2.5 External Interface Requirements 44
2.5.1 User interface Requirement 44
2.5.2 Software interfaces 44
2.5.3 Hardware interfaces 45
2.5.4 Communication Interfaces 45
2.6 Summary 45
Design and Architecture 46
3. System Design 47

3.1 Design Considerations 47
3.2 Design Models 48
3.2.1 Class Diagram 48
3.2.3 Sequence diagram 49
3.1.4 State Transition Diagram 52
3.2 Architectural Design 55
3.2.1 Component Diagram 55
3.2.2 N-Tier Architecture 56
3.3 Data Design 56
3.4 Behavioral Model 62
3.4 Design Decisions 68
3.5 User Interface Design 69
3.5.1 Screen Images 69
3.5.2 Screen Objects and Actions 70
3.5 Summary 70
Implementation 72
4.1 Algorithms 72
Algorithm 5 – Manage Stock 76
4.2 External APIs/SDKs 80
4.3 Code Repository 81
4.4 Summary 81
Testing And Evaluation 82
5.1 Unit Testing 83
5.1.1 Test case1: 83
5.1.2 Test case2: 83
5.1.3 Test case3: 84
5.1.4 Test case4: 84
5.1.5 Test case5: 85
5.1.6 Test case6: 85
5.1.7 Test case7: 86
5.1.8 Test case8: 86
5.2 Functional Testing 87
5.2.1 Test case 1: 87

5.2.2 Test case 2: 87
5.2.3 Test case3: 88
5.2.4 Test case4: 89
5.2.5 Test case5: 89
5.2.6 Test case6: 90
5.2.7 Test case 7: 90
5.2.8 Test case 8: 91
2.3 Integration Testing....91
5.3.1 Test Case 1 92
5.3.2 Test Case 2 92
5.3.3 Test Case 3 93
5.3.4Test Case 4 93
5.3.4 Test Case 5 94
5.3.5 Test Case 6 94
5.4 Performance Testing....95
5.4.1 Test Case 1: 95
5.4.2 Test Case 2: 95
5.4.3 Test Case 3: 96
5.4.4 Test Case 4: 97
1.5 Summary....97
System Conversion....98
6 Introduction: 99
6.1 Conversion Method: 99
6.1.1 Pilot Conversion:....99
6.2 Deployment:....99
1.2.1 Data Conversion: 100
6.2.2 Training: 100
6.3 Post Deployment Testing: 101
6.4 Challenges and solutions: 101
6.5 Summary: 102
Conclusions....103
Introduction....104
7.1 Evaluation....104

7.2 Traceability Matrix 105
7.3 Conclusion 106
7.4 Future Work 106
References 107
Appendix-A Use case Description 109
Appendix B: General Coding Standards & Guidelines 120
Appendix C: Application Prototype 125

List of Tables

Table 1-1: Related Review work....22
Table 1-2 :Tools, Technologies and libraries....23
Table 2-1:User Classes and characterization....26
Table 2-2: Use Case 1....26
Table 2-3: Use case 2....27
Table 2-4: Use case 3....28
Table 2-5: Use case 4....28
Table 2-6: Use case 5....29
Table 2-7: Use case 6....30
Table 2-8: Use case 7....30
Table 2-9: Use case 8....31
Table 2-10: Use case 9....31
Table 2-11: Use case 10....32
Table 2-12: Use case 11....32
Table 2-13: Use case 12....33
Table 2-14: Use case 13....33
Table 2-15: Use case 14....34
Table 2-16: Use case 15....35
Table 2-17: Use case 16....35
Table 2-18: Functional requirement 1....37
Table 2-19: Functional requirement 2....37
Table 2-20: Functional requirement 3....38
Table 2-21: Functional requirement 4....38
Table 2-22: Functional Requirement 5....38
Table 2-23: Functional requirement 6....39
Table 2-24: Functional requirement 7....39
Table 2-25: Functional requirement 8....40
Table 2-26: Functional Requirement 9....40
Table 2-27: Functional Requirement 10....40
Table 2-28: Functional requirement 11....41
Table 2-29:Functional requirement 12....41
Table 2-30: Functional requirement 13....41
Table 2-31: Functional Requirement 14....42
Table 2-32: Functional Requirement 15....42
Table 2-33: Functional requirement 16....43
Table 2-34: Functional Requirement 17....43
Table 2-35: Non-Functional Requirements....44
Table 3-1: Data Dictionary....59
Table 4-1 Algorithm 1....72

Table 4-2 Algorithm 2 73
Table 4-3 Algorithm 3 74
Table 4-4 Algorithm 4 74
Table 4-5 Algorithm 5 76
Table 4-6 Algorithm 6 76
Table 4-7 Algorithm 7 77
Table 4-8 Algorithm 8 78
Table 4-9 Algorithm 9 79
Table 5-1: Unit test Case 1 83
Table 5-2: Unit test case 2 83
Table 5-3: Unit test case 3 84
Table 5-4: Unit test case 4 84
Table 5-5: Unit test case 5 85
Table 5-6: Unit test case 6 85
Table 5-7: Unit test case 7 86
Table 5-8: Unit test case 8 86
Table 5-9: Functional test case 1 87
Table 5-10: Functional test case 2 87
Table 5-11: Functional test case 3 88
Table 5-12: Functional test case 4 89
Table5-13: Functional test case 5 89
Table 5-14: Functional test case 6 90
Table 5-15: Functional test case 7 90
Table 5-16: Functional test case 8 91
Table 5-17: Integration test case 1 92
Table 5-18: Integration test case 2 92
Table 5-19: Integration test case 3 93
Table 5-20: Integration test case 4 93
Table 5-21: Integration test case 5 94
Table 5-22: Integration test case 6 94
Table 5-23: Performance test case 1 95
Table 5-24: Performance test case 2 95
Table 5-25:Performance Test case3 96
Table 5-26: Performance Test case 4 97
Table 6-1:Deployment 99
Table 6-2: Data Conversion 100
Table 6-3: Instruction 1 100
Table 6-4: Instruction 2 100
Table 6-5: Post Deployment testing 101
Table 6-6: Challenges and solutions 102
Table 7-1 Objectives with status 104

Table 7-2: Traceability matrix.....105
Table 7-3: Objectives with functionalities 106

List of figures

Figure 1-1: Gantt Chart 24
Figure 2-1 Use Case Diagram 36
Figure 3-1 Class Diagram 48
Figure 3-2 Sequence Diagram 1 49
Figure 3-3 Sequence Diagram 2 50
Figure 3-4 Sequence Diagram 3 51
Figure 3-5 Sequence Diagram 4 52
Figure 3-6 State Diagram 1 53
Figure 3-7 State Diagram 2 53
Figure 3-8 State Diagram 3 54
Figure 3-9 State Diagram 4 54
Figure 3-10 Component Diagram 55
Figure 3-11 Architectural Diagram 56
Figure 3-12 ER Diagram 58
Figure 3-13 Screen Images 69
Figure 3-14Screen Objects & Action 70

Chapter 1

Introduction

Introduction

This chapter introduces the website kid's vaccination booking system that is designed to maintain the immunity of child. This platform is designed to give proper awareness to users about importance of vaccine. The website ensures that users are directly connected with hospitals. By addressing the existing websites this platform shows the efficient and reliable booking system in the desired hospitals.

1.1 Problem statement

Many people have not proper knowledge about importance of vaccination. Sometime the vaccination cards can be lost or misplaced and parents face difficulty to find it whenever there is need. Vaccination centers are not available in some areas. Parents do not know which vaccine should be given to the child at which age of child. Due to this some important doses of vaccination like measles, cancer, malaria etc. are missed. And such diseases can occur whose treatment is very expensive. In Pakistan many children are died from the diseases which can be preventable by vaccination.

1.2 Problem solution

This platform is designed to give proper awareness to users about importance of vaccine. This website ensures that children should be vaccinated with a suitable vaccine based on their ages. The website ensures that users are directly connected with hospitals. The website is user-friendly. It also provides the auto scheduling where users can get the vaccine schedule automatically based on their child ages. Users can also use the available slots of hospitals. The website enables users to get timely reminders through SMS/E-mail and saving the previous record. There is a separate admin panel where admin verify the users and hospitals approval, manage reviews and monitor overall system.

1.3 Objectives

- Design a platform where users can book and take appointments for their child vaccination in desired hospitals.
- Allow hospitals to post detailed information about vaccine stock that might helpful for parents in booking.
- Provide more hospital options to parents for improving their child vaccination.
- Establish a simple and well-organized booking system.
- Make a user-friendly platform.
- Provide the facility of Govt. services for the people who can't afford private dues.
- Provide the facility of saving previous records.
- Gain trust in parents through verification for team.
- Users can see hospital's charges when they select the private hospitals.

1.4 Scope

The scope of this project is to develop a user-friendly website that provide the facility of vaccination only for kids. This platform involves features like provide notification for the remembrance of parents, auto scheduling through which they can get the suitable vaccine according to age automatically, real time slotting for the selection of available hospitals and securely save the record after vaccination. This user-friendly platform is developed through MERN stack (MongoDB, Express.js, Node.js and React.js).

Stakeholders

- Parents
- Kids
- Hospitals/Health Workers
- Admin

Limitations

- It is only for kids.
- Requires fast internet connection.
- This platform does not provide the exact location in real time slotting.
- The system slows down when it used by large number of clients.
- This platform does not provide payment process.

1.5 System Components

The website will have three panels (parents, hospital and admin). It requires stable internet connection, if the user has not internet connection, then they cannot access the website. This website has the following features with their panels:

1. Module 1: Home page

This module shows the home screen that allows hospital and parents to register for accessing their dashboards. The end users also view the vaccine info and vaccine guidelines.

Features

- **Vaccine info:**

This gives a complete general vaccine information according to the child's age.

- **Guidance:**

This feature gives the guidance and tells the importance of vaccine for their awareness.

- **Hospital Signup:**

In this feature the hospital will enter his personal details like hospital name, e-mail, password, certificate. After this hospitals account will be created and redirected to login page.

- **Parent Signup:**

In this feature system displays a splash screen where parent enter their credentials e-mail, name, address, contact number and password. After validation user is navigated too login age.

- **Login:**

User will enter e-mail and passwords and system verify its role through its details and display the suitable screen.

- **Forget password:**

If the user will forget their password this feature ask user to enter their e-mail and system sent the code, by entering this code user can access their panels.

2. Client Web App (Parent panel)

This panel allow users to Add child for taking appointments by entering their requires details, view the vaccine schedule and records. Parents can also add multiple children under one registration and view their records separately.

Features

- **Add child:**

In Add child form parent will enter child information like name, DOB, address, select hospital from available slots, select service type(govt/private), request type (Home/Hospital), upload a picture and birth certificate.

- **Vaccine Overview:**

In this feature, parents can view the whole vaccine schedule of children upcoming vaccine, completed vaccine number of children. Parent can also view the vaccine record of their child that has been done and also view the hospital name through which it is done.

- **Setting:**

In this feature users can update their profile by changing their password, name or any other information.

3. Hospital WebApp (Hospital panel)

In this panel system allow the hospitals to redirect in the system and provide the services to the users.

Features

- **Manage stock:**

In vaccine stock, hospital manage/ update the stock.

- **Manage Schedule:**

In Vaccine schedule Hospital can manage the schedule of child by entering the due date and marking the vaccine as given once the child receives it.

- **Appointment:**

This feature allows hospitals to manage appointments like approve or reject the appointments that is booked by parents.

- **Manage slots:**

In this feature the hospitals manage their slots based on their availability in a day.

- **View Rating:**

In this feature Hospital can view the rating submitted by users.

- **Setting:**

In this feature hospitals can update their profile by changing their password, name or any other information.

4. Admin Web App (Admin Panel)

This panel manage the overall system by managing the hospitals and users request, have authority to remove the hospitals based on their performance.

Features

- **View Rating:**

In this feature the admin can view the rating that is submitted by users. And admins can take action based against hospitals based on the rating.

- **Manage Hospitals:**

This feature enables admin to approve or reject hospitals request.

- **Manage Parents:**

This feature enables the admin to view or delete the parent.

- **Setting:**

In this feature admin can update their profile by changing their password, name or any other information.

5. Backend API Server (Node.js + Express)

- Restful APIs for all client interactions
- User authentication and authorization
- Appointment scheduling logic and availability checks
- Storage APIs
- Secure access control for data

6. Database (MongoDB)

- Store user information (patients and Hospitals)
- Appointment details and history
- Vaccine records
- All the system details.

1.5 Related System Analysis/ Literature Review

This project focuses on the vaccination of kids. By searching the existing website no other website provide all features in one platform.

Table 1-1: Related Review work

Websites name	Weakness	Proposed solution
E-Shifa	Allow users to get vaccinate at home but in limited areas. It also lacks of auto scheduling.	The proposed solution provides the facility to get vaccinate from everywhere. It also generates the schedule automatically based on child DOB.
Vaccine.pk	Not allow users to get vaccination from their desired hospitals. Can't manage multiple children under a parent registration	This platform allows users to book in desired hospitals. Add multiple children under one parent registration.

1.7 Vision statement

Here is a vision statement for Kids vaccination booking system:

For [Patient, Hospitals, Healthcare centers]

Who [need a way to efficiently manage children vaccine course]

The [Kids Vaccination Booking]

Is [a website for online booking and taking appointments for kids]

That [allows users to book appointment, get reminders, save records, help parents to complete child's vaccine schedules timely]

Unlike [Existing system or other annual booking websites]

Our Product [Provide a user friendly, secure, reliable platform with simple appointment selection]

1.8 System Limitations and Constraints

Limitations:

- It is only for kids.
- Requires fast internet connection.
- This platform does not provide the exact location in real time slotting.
- The system slows down when it used by large number of clients.
- This platform does not provide payment process.

Constraints:

- **Scalability:** Parents will book the appointment of one child in one hospital at a time before confirmation.
- **Performance:** The website will load in 3-4 seconds.
- **Technical:** Stable internet will be required.

- **Security:** Users will have to login by their passwords for viewing their dashboards.

1.9 Tools and Technologies

TOOLS, LIBRARIES AND TECHNOLOGIES WITH REASONIING

Table 1-2 :Tools, Technologies and libraries

Tools, Technologies and libraries	**Tools**	**Version**	**Rational**
	VS code	1.106	An IDE for running and testing code
	npm	11.6.3	For installing libraries
	Libraries	**version**	**Rational**
	Mongoose atlar	8.20.0	For connecting Backend to Database.
	Axios	1.13.2	For Calling Backend APIs.
	Multer cloudinary	2.0.2	For uploading pictures.
	JWT	9.0.2	Used for authentication
	lifetimesms	5.10.6	For Reminders through SMS
	nodemailer	8.0.6	For send e-mails
	Technologies	**Version**	**Rational**
	Express.js	5.1.0	Handles APIs
	React.js	19.2	Frontend UI (Schedules, Dashboards etc.)
	Node.js	v25.2.1	Execute server-side JavaScript and run APIs
	MongoDB	8.2.2	For storing data in database (NO SQL).

1.10 Work division

Sr. No	Roll Numbers	Names	Role Assignment and Work Division
1.	077007	Sehar Younas	Frontend +Backend+ Documentation
2.	077025	Hifza Shahzad	Frontend +Backend+ Documentation

1.10 Project Deliverables

Our project will have followings deliverables (7th Semester)

1. Project Proposal
2. Requirement Traceability Matrix (RTM)
3. System Design Specification (SDS)
4. Prototype

Our project will have following deliverables (8th Semester)

1. Coding
2. Testing
3. Deployment
4. Final Evaluation

1.11 Project Planning

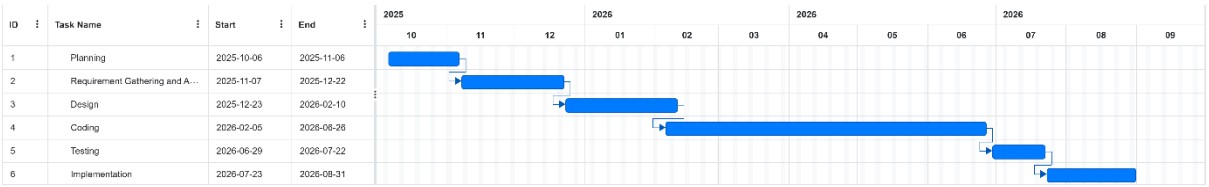


Figure 1-1: Gantt Chart

1.12 Summary

In this chapter, objectives and goals of project Kids vaccination booking are defined. This section also shows the project deliverables and how the project achieves its objectives and goals within the planned time to maintain its scope.

Chapter 2

Requirement Analysis

2 Analysis

This chapter focuses on identifying the requirements (functional and non- functional). It also analyzes that who will use this platform based on their perspectives. All the use cases and their rationale are identified here.

2.1 User classes and characteristics

Table 2-1:User Classes and characterization

User Classes	Characterization
Parent	Parent is an end user that can book and take appointments, get reminders select the available slots and view hospital profile after login.
Hospital	Hospital could be a user that manages vaccine stock, schedule and slots that accept or reject the requests of parent.
Admin	Admin is a specific person that manages and monitor the overall system like users, hospital's request and view rating and remove the hospital based on rating.

2.2 Requirement Identifying Techniques

These are the basic techniques that are used to identifying requirements. These are as follows:

2.2.1 Interviews

Interviews were conducted with stakeholders which involve parent hospitals and different health workers of immunization centers to gather information about their requirements/need and problems.

2.2.2 Ethnography and Observation

By using the technique, it was observed how people force difficulty in maintaining their child's vaccine course, tracking their vaccines schedule and taking appointment physically in desired hospitals. Through observation the requirement of the system identified like giving reminders timely, vaccine record and taking online appointments.

2.2 Use Case Analysis

This technique describes the interaction between user and the system. The user uses the system with their own perspective, which help in identifying functional requirements.

2.2.1 Use case 1

Table 2-2: Use Case 1

ID	UC-1
Name	Guidance

Actors	End users
Description	This use case gives a brief description about the importance of vaccines
Trigger	User select the Guidance on home page
Preconditions	User must have access/open the website
Post condition	A complete guidance is shown on the screen User can read it.
Normal flow	1. User open the website. 2. Select the “Guidance” option on homepage. 3. Guidance screen appears.
Alternative flow	If the user has not stable network. Then the website will not show the page and msg show please check your network connection.
Business flow	Data us fetched from the database and shown on screen.
Assumptions	User must have stable internet connection. Database must be active for fetching and displaying data.

2.2.2 Use case 2

Table 2-3: Use case 2

ID	UC-2
Name	Vaccine info
Actors	End users
Description	This use case gives a brief description about the doses of vaccines
Trigger	User select the Vaccine info on home page
Preconditions	User must have access/open the website
Post condition	A complete vaccine schedule is shown on the screen User can read it.
Normal flow	1. User open the website. 2. Select the “Vaccine info” option on homepage. 3. Guidance screen appears.
Alternative flow	If the user has not stable network. Then the website will not show the page and msg show please check your network connection.
Business flow	System fetched from the database and shown on screen.
Assumptions	User must have stable internet connection. Database must be active for fetching and displaying data.

2.2.3 Use case 3

Table 2-4: Use case 3

ID	UC-3
Name	Parent Signup
Actors	End users
Description	This use case allows users to enter their email, strong password and their number
Trigger	The user opens the website and c lick on parent sign up on the screen.
Preconditions	User should not be registered already
Post condition	A new user account is created in database.
Normal flow	1. User open the website. 2. A dashboard is appeared. 3. User click the parent sign up. 4. User enter their credentials like e-mail, name, phone number and a strong password 5. Anew account is created in database. 6. User is redirected to the login screen.
Alternative flow	If the e-mail is registered, an error message will appear. If the user does not enter a strong password message shown to correct it.
Business flow	E-mail must be unique. Password must be unique like a mix of numbers/ characters, special numbers.
Assumptions	User must have internet access.

2.2.4 Use case 4

Table 2-5: Use case 4

ID	UC-4
Name	Hospital Signup
Actors	End users
Description	This use case allows users to enter their email, strong password and their number
Trigger	The user opens the website and click on Hospital sign up on the screen.
Preconditions	User should not be registered already.
Post condition	A new hospital account is created in database.

Normal flow	1. User open the website. 2. A dashboard is appeared. 3. User click the hospital sign up. 4. User enter their credentials like e-mail, name, phone number, a strong password, certificate, slots and address. 5. A new account is created in database. 6. User is redirected to the login screen.
Alternative flow	If the e-mail is registered, an error message will appear. If the user does not enter a strong password message shown to correct it.
Business flow	E-mail must be unique. Password must be unique like a mix of numbers/ characters, special numbers.
Assumptions	Hospital must have internet access.

2.2.5 Use case 5

Table 2-6: Use case 5

ID	UC-5
Name	Login
Actors	User
Description	This use case allows users to login e-mail and password.
Trigger	Users click on the login on the dashboard.
Preconditions	User must be already registered.
Post condition	If the e-mail and password is correct user redirected to their panel. If e-mail and password is invalid an error appears.
Normal flow	1. User open the website a dashboard appears. 2. Click on the login features. 3. Enter email and password. 4. If valid, user can access to their panel
Alternative flow	If the email is invalid an error message is shown.
Business flow	Email and password must be valid. User must be registered already.
Assumptions	User must have stable internet connectivity. User must have knowledge.

2.2.6 Use case 6

Table 2-7: Use case 6

ID	UC-6
Name	Forgot password
Actors	End users
Description	This use case allows users to reset their password via email
Trigger	User click on the forgot password.
Preconditions	User must be registered.
Post condition	User reset passwords and can get access to their panel.
Normal flow	1. User click on “forgot password”. 2. User enter register email. 3. A code is sent to the email. 4. User enter this code and then user redirected to their dashboard.
Alternative flow	If the user enters the unregistered email, then an error message will show.
Business flow	Email must be valid.
Assumptions	User must have stable internet connection.

2.2.7 Use case 7

Table 2-8: Use case 7

ID	UC-7
Name	Setting
Actors	End users
Description	This use case allow user to change their name and passwords.
Trigger	User click on the setting on their dashboard.
Preconditions	User must already be registered
Post condition	Users’ data is undated.
Normal flow	1. Users tap on “setting”. 2. User tap on “edit” 3. A screen appears. 4. User update his information. 5. Updated information is stored in database.
Alternative flow	If the internet connection is not stable then data is not updated and message “check your connection” is shown.

Business flow	The updated data must be validated.
Assumptions	User must have stable internet connection. User must be already registered.

2.2.8 Use case 8

Table 2-9: Use case 8

ID	UC-8
Name	Add child
Actors	End user
Description	This use case allows users to register their child by filling form and entering required child's credentials
Trigger	User select the "Add Child" on the dashboard.
Preconditions	Parent must be login into the system.
Post condition	Child is register and schedule is created automatically.
Normal flow	1. User tap on "Add Child" 2. A child form appears 3. User enters child's name, DOB, select service type, select the hospital from dropdown, click on auto schedule, upload child's picture and birth certificate. 4. Click on submit.
Alternative flow	If user enter incomplete information, then the error message shown "Enter complete information" If the child is already registered then error message shown.
Business flow	Childs data must be accurate.
Assumptions	User has stable internet connection for submitting child's form

2.2.9 Use case 9

Table 2-10: Use case 9

ID	UC-9
Name	Vaccine Overview
Actors	End user
Description	This use case allows users to view schedule and record of their child vaccination course.
Trigger	User select the "Vaccine Overview" on dashboard.
Preconditions	Child must be registered.
Post condition	The complete child's vaccine record is displayed with upcoming and completed vaccine.

Normal flow	1. Users tap on the “Vaccine Overview” 2. A screen is displayed with complete vaccination course. 3. User can view the upcoming vaccine with date.
Alternative flow	If the child is not registered then the schedule is not displayed. If the hospital does not update the schedule, then the schedule is snot updated.
Business flow	Child vaccine data is updated by the hospital. This stored in database Database update data in parents’ schedule And displayed to the user.
Assumptions	Data must be updated by hospital. Database must be active to show the updated data

2.2.10 Use case 10

Table 2-11: Use case 10

ID	UC-10
Name	Manage stock
Actors	End user (Hospital)
Description	This use case allows hospitals to enter their vaccine stock. Hospital can also update it.
Trigger	Hospitals click on the “Manage Stock”” on their dashboard.
Preconditions	Hospitals must be registered and login.
Post condition	Vaccine stock screen displayed where the hospitals enter their stock.
Normal flow	1. Hospital taps on “Manage stock” 2. Vaccine stock screen appears 3. Hospitals enter vaccine stock details 4. This data is stored in database.
Alternative flow	If the internet is not stable then data is not stored in database.
Business flow	System store data in database after entering the data.
Assumptions	Hospitals must have stable internet connection Database must be activated to store data.

2.2.11 Use case 11

Table 2-12: Use case 11

ID	UC-11

Name	View Rating
Actors	Hospital
Description	This use case allows hospitals to view the rating submitted by users.
Trigger	Hospital clicks on View Rating on the dashboard.
Preconditions	Hospital must be login and rating is submitted by parents.
Post condition	A screen appeared where all the stock of vaccine is shown.
Normal flow	1. Hospital taps on the vaccine stock. 2. A screen of rating appears. 3. Hospital can view it
Alternative flow	If user enter invalid data (negative number) the system shows error message.
Business flow	Accurate data must be entered.
Assumptions	User must have already stock.

2.2.12 Use case 12

Table 2-13: Use case 12

ID	UC-12
Name	Manage Slots
Actors	Hospital
Description	This use case allows hospitals to manage their slots.
Trigger	Hospital clicks on manage slots on the dashboard.
Preconditions	Hospital must be registered and logged in.
Post condition	A screen appeared where hospital can add their slots.
Normal flow	1. Hospital taps on the manage slots. 2. Hospital enters No. of slots and days availability. 3. These new slots are stored in database 4. Hospital can also update it.
Alternative flow	If user enter invalid data (negative number) the system shows error message.
Business flow	Accurate data must be entered.
Assumptions	Hospitals must have stable internet connection Database must be activated to store data.

2.2.13 Use case 13

Table 2-14: Use case 13

ID	UC-13

Name	Manage Appointment
Actors	Hospitals
Description	This use case allows users to manage the parents' appointment request.
Trigger	Hospital clicks on "manage appointment".
Preconditions	Hospital must be login the system.
Post condition	A screen with a lot of appointments displayed.
Normal flow	1. Hospital admin clicks on manage appointments. 2. A list of appointment / request is displayed on the screen 3. Hospital admin can approve or reject the request. 4. The system updates the status in the database. 5. Database send notification to user regarding decision.
Alternative flow	If the internet is not stable, status is not updated in database. And database does not send notification.
Business flow	System must update data in database.
Assumptions	Appointment/ request must be submitted by the users Hospital must have stable internet connection.

2.2.14 Use case 14

Table 2-15: Use case 14

ID	UC-14
Name	Manage schedule
Actors	Hospitals
Description	This use case allows hospital to manage the child's vaccine schedule.
Trigger	Hospital clicks on the vaccines schedule on dashboard.
Preconditions	Hospitals must be logged in. Childs vaccine stock must be existed already in the system
Post condition	Hospitals updated the schedule and data is stored in the database.
Normal flow	1. Hospital admin taps on manage schedule 2. Schedule of multiple children is displayed on screen. 3. If vaccine is given staff marked it read as completed 4. Hospital staff enter the next due date for upcoming vaccine. 5. System saves the updated data successfully.
Alternative flow	If the staff enter invalid data (past date) then system shows error message If the network is not stable then updated data is not stored in database.

Business Flow	System stored the updated d ata in database. System automatically updates the child's vaccine schedule after changes made by hospital.
Assumptions	Hospital must have permissions to change the schedule.

2.2.15 Use case 15

Table 2-16: Use case 15

ID	UC-15
Name	Manage hospital
Actors	Admin
Description	This use case allows users to approve or reject hospitals request
Trigger	Admin tap on the manage hospitals on their dashboard.
Preconditions	Admin must be logged in.
Post condition	Admin verified the hospital and granted request to the system.
Normal flow	1. Admin tap on manage hospitals request. 2. A screen with the hospital request will display. 3. Admin made a decision through verification. 4. System stores the decision approve or reject in the database.
Alternative flow	If the internet connection is not stable, decision is not stored in the database.
Business flow	Hospital data must be validated before granting access.
Assumptions	Internet connection must be available during verification.

2.2.16 Use case 16

Table 2-17: Use case 16

ID	UC-16
Name	Manage Rating
Actors	Admin
Description	This use case allows admin to evaluate hospital performance and made decision based on the performance.
Trigger	The admin clicks on the view rating on their dashboard.
Preconditions	Admin must be logged in in the system.
Post condition	Admin evaluating the hospital performance through users rating.

Normal flow	1. Admin taps on manage hospitals performance. 2. A screen with hospital along with their rating displayed. 3. Admin evaluate the performance. 4. Deactivate the hospital based on their performance if the rating is below the 3 stars. 5. System stores the data in the database.
Alternative flow	If no users rating is available admin will not be able to evaluate the performance.
Business flow	System stores the updated data. System sends the notification the hospitals.
Assumptions	User rating must be existed in the system.

2.2.17 Use case Diagram



Figure 2-1 Use Case Diagram

Description

Use case diagram shows how a system interacts with system to achieve goals. This use case diagram shows three users (Parent users, Hospital, Admin). That can interact with system to achieve desire goals. User interact to add child, view record and schedule by Sign-up and login. Hospital interacts to manage stock and slots and view appointment. Hospitals also need to register themselves into the system. Admin can manage the overall system by login into the system.

2.4 Functional Requirements

This section describes the functional requirements of overall project. The requirements are defined according to the panel with its features.

2.4.1 Functional Requirements 1

Table 2-18: Functional requirement 1

Identifier	FR1
Title	Guidance
Requirement	The user will able to view the guidance about the vaccine
Rational	For awareness in people about importance of vaccine.
Sources	Business Requirement
Business	The parent will have the internet and browsers to view it.
Dependencies	Fast internet connection
Priority	Medium

2.4.2 Functional Requirements 2

Table 2-19: Functional requirement 2

Identifier	FR2
Title	Vaccine info.
Requirement	The user will view the overall vaccine details
Rational	For knowing that which vaccine is done at which age.
Sources	Business Requirement
Business	The user will have internet and browsers to visit it.

Dependencies	Fast internet connection
Priority	Medium.

2.4.3 Functional Requirements 3

Table 2-20: Functional requirement 3

Identifier	FR3
Title	Parent Signup
Requirement	The parent will be able to register by entering name uses name, password and contact no. and then submit it
Rational	For taking and booking appointment for their child.
Sources	Business Requirement
Business	The parent will be registered before login
Dependencies	Fast internet connection
Priority	High

2.3.4 Functional Requirement 4

Table 2-21: Functional requirement 4

Identifier	FR4
Title	Hospital Signup
Requirement	The hospital will be able to register by entering name-mail, password, contact No. and certificate then submit it
Rational	For Hospital registration and providing services.
Sources	Business Requirement
Business	Hospital must be registered before login.
Dependencies	Fast internet connection
Priority	High

2.3.5 Functional Requirement 5

Table 2-22: Functional Requirement 5

Identifier	FR5
Title	Login

Requirements	The end user (parent, hospital) will able to login by entering email or password.
Rational	For authentication based on their roles.
Sources	Business Requirement.
Business Role	Password must be secure and unique.
Dependencies	FR-3, FR-4
Priority	High

2.3.6 Functional Requirement 6

Table 2-23: Functional requirement 6

Identifier	FR-6
Title	Forgot Password
Requirements	The end user will be able to reset passwords if he forgets it.
Rationale	For accessing dashboard in case of forgetting password.
Sources	Business requirement
Business	The entered e-mail must be accurate.
Dependencies	FR-3, FR-4.
Priority	High

2.3.7 Functional Requirement 7

Table 2-24: Functional requirement 7

Identifier	FR-7
Title	Setting
Requirements	All the users (admin, hospitals and parent) navigate to the setting in their dashboards and reset the passwords
Rationale	For updating profile
Sources	Business requirement
Business	The parent must be registered before login
Dependencies	FR-3, FR-4, FR-5
Priority	High

2.3.8 Functional Requirement 8

Table 2-25: Functional requirement 8

Identifier	FR-8
Title	Addchild
Requirements	The user will be able to add child by entering name, B. form, gender, DOB.
Rational	For taking and booking appointments in hospitals for their child.
Sources	Business requirement
Business	Book one hospital till confirmation.
Dependencies	FR-3, FR-4
Priority	High

2.3.9 Functional Requirement 9

Table 2-26: Functional Requirement 9

Identifier	FR-9
Title	Auto scheduling
Requirements	Parent will able to view the vaccine names.
Rational	For tracking the vaccines schedule automatically.
Sources	Business requirement
Business	Parent must enter the correct DOB.
Dependances	FR-3, FR-5, FR-8
Priority	High

2.3.10 Functional Requirement 10

Table 2-27: Functional Requirement 10

Identifier	FR-10
Title	View Hospital profile/Rating
Requirements	Parents will also view the hospital profile and rating.
Rational	For booking in hospital that may not cause of frustration.

Sources	Business requirement
Business	Rate at least three stars in case of satisfaction.
Dependances	FR-3, FR-5, FR-8
Priority	High

2.3.11 Functional Requirement 11

Table 2-28: Functional requirement 11

Identifier	FR-11
Title	Vaccine record
Requirements	Parent will able to view the vaccine record of child
Rational	For track the previous doses timely.
Sources	Business requirement
Business	Child must have at least on dose.
Dependances	FR-3, FR-5, FR-8
Priority	High

2.3.12 Functional Requirement 12

Table 2-29:Functional requirement 12

Identifier	FR-12
Title	Vaccine schedule
Requirements	Parent will able to view the vaccine schedule of child
Rational	For viewing the upcoming vaccine and received vaccine timely.
Sources	Business requirement
Business	Parent must be registered and take appointment for at least on child.
Dependances	FR3, FR-5, FR-8
Priority	High

2.3.13 Functional Requirement 13

Table 2-30: Functional requirement 13

Identifier	FR-13
Title	Manage Stock
Requirements	Hospital will entering their available vaccine stock.
Rational	For managing the vaccines stock.
Sources	Business requirement
Business	The hospital must enter the correct details.
Dependances	FR-4, FR-5
Priority	High

2.3.14 Functional Requirement 14

Table 2-31: Functional Requirement 14

Identifier	FR-14
Tittel	Manage Slots
Requirements	Hospital will entering their available slots.
Rational	For managing slots and providing services.
Sources	Business requirement
Business	Hospital must enter the correct details.
Dependances	FR-4, FR-5
Priority	High

2.3.15 Functional Requirement 15

Table 2-32: Functional Requirement 15

Identifier	FR-15
Title	Manage Appointments
Requirements	Hospital will be tapping reject and accept the appointments.
Rational	For managing the request
Sources	Business requirement
Business	Hospital will approve the request if it is available.

Dependances	FR-3, FR-4, FR-8
Priority	High

2.3.16 Functional Requirement 16

Table 2-33: Functional requirement 16

Identifier	FR-16
Title	Manage Hospitals
Requirements	Admin will approve or reject the hospitals
Rational	For managing the hospitals and Parent request.
Sources	Business requirement
Business	Approve request based on the certificates and parent details
Dependances	FR-3, FR-8.
Priority	High

2.3.17 Functional Requirement 17

Table 2-34: Functional Requirement 17

Identifier	FR-17
Title	Manage Rating/reminders
Requirements	Admin will be able to monitor the rates against hospitals
Rational	For making the friendly platform and takes action against hospitals and notify to the hospitals
Sources	Business requirement
Business	Hospitals must show satisfactory performance
Dependances	FR-4, FR-10
Priority	High

2.4 Non-Functional Requirements

Non-Functional requirements are the constraints and rules on the services and functions that are offered by the system Thess are implemented on the overall system rather than the single module. These include constraints on performance and availability of website. The non-Functional Requirements are as follows.

Table 2-35: Non-Functional Requirements

Identifier	Category	Description
NFR-1	Performance	Website should load the home screen in 2-3 seconds.
NFR-2	Performance	Response is given in 3-5 seconds after submitting form
NFR-3	Useability	System is user friendly and it provide the user interface and new user should take 5 minutes to easily understand it
NFR-4	Scalability	It will applicable for thousands of people but one parent can take appointment till confirmation.
NFR-5	Reliability	System is reliable because users can save the record and do not lose their data.
NFR-6	Security	The system is accessed only by the authorized person and resist to unauthorized attempts.

2.5 External Interface Requirements

External interface requirement describes how the system interact with the external interfaces like users, hardware devices, software and communication networks. These also describe how the data is exchanged and how different interfaces behave.

2.5.1 User interface Requirement

The system Kids Vaccination booking will provide a user-friendly interface for users:

- The system will be a website that is accessed through internet connection.
- The interface will be very smooth and light colors like (green, blue and grey) appears.
- The buttons like save, delete, upload, submit reject and approve are all available.
- System also displays the error messages. (form is submitted successfully).
- System interface also displays the navigation bars on the pages.
- The page is accessible and responsive on laptop, computer and tablet.
- Clear and effective font style will display.

2.5.2 Software interfaces

The system will interact with the following software interfaces:

- Database is MongoDB for storing the data (child's record, hospital record, vaccine schedule, history etc.)

- Operating system is windows.
- Web browsers are required like google, chrome etc.
- External APIs Twilio node mailer is used for the SMS or email to notify users.

2.5.3 Hardware interfaces

The system will require the following hardware interfaces:

- Desktop tablet and mobile are required for users (admin, parent hospital).
- Keyboard, mouse and touch screen is also required for input data.
- Display screen is required for viewing output.
- This system does not require any special hardware.

2.5.4 Communication Interfaces

The system will support the following communication interfaces:

- Fast internet connection is required.
- The system will send an e-mail notification before vaccination date.
- The system will also send SMS for notify to the users.

2.6 Summary

This section analyzes the requirements through the use case analysis, interview and observations according to the need of parents and hospitals. In this chapter all the external requirements like user, communication, hardware and software are analyzed.

Chapter 3

Design and Architecture

3. System Design

System design involves product perspective, in which platform will operate including dependencies and interaction with other system. It also describes the design constraints like performance.

Product Perspectives:

Kids Vaccination System is a website that allow users to book and take appointments in desired hospitals. The designed system run in health care industry, developing a website where user directly connect with hospitals and immunization centers.

Dependencies:

Frontend (React.js): This is used for making the interfaces of user.
Backend(node.js+express.js): Depends on it for handling all the backend system and for connecting database and backend.
Database MongoDB: Used to store data and retrieved when required.

Interaction with other System:

Reminder: System interact with the services SMS/ E-mail for sending reminders.
Device compatibility: The system interacts with browsers of android/ laptop and PC to run the system.

Design Constraints:

Scalability: Parents will book the appointment of one child in one hospital at a time before confirmation.
Performance: The website will load in 3-4 seconds.
Technical: Stable internet will be required.
Security: Users will have to login by their passwords for viewing their dashboards.

3.1 Design Considerations

Assumption and dependencies:

- The developing website will run on android version 7.0/laptop and PC.
- He required RAM is 2GB to run this website.
- Website will require the stable internet connection.
- Web-Browsers are required to run it.

Limitations:

- It is only for kids.
- Payment system will not include in it.
- This platform does not provide the exact location in real time slotting.
- The system slows down when it used by large number of clients.

Risks:

- Booking in multiple centers for same child. Display the slots unavailable to many others.

Mitigation: Users will book one team or hospital at a time till confirmation message.

- If hospitals will not work properly, it will cause frustration for users.

Mitigation: The user can rate hospitals and action will be taken against hospitals

3.2 Design Models

3.2.1 Class Diagram

Class diagram shows the classes, attributes, methods and the relationship between classes. This diagram represents different classes user, parent, hospital, admin, vaccine stock, appointment. And each class has its own attributes and function. Each class shows a relation with other class.

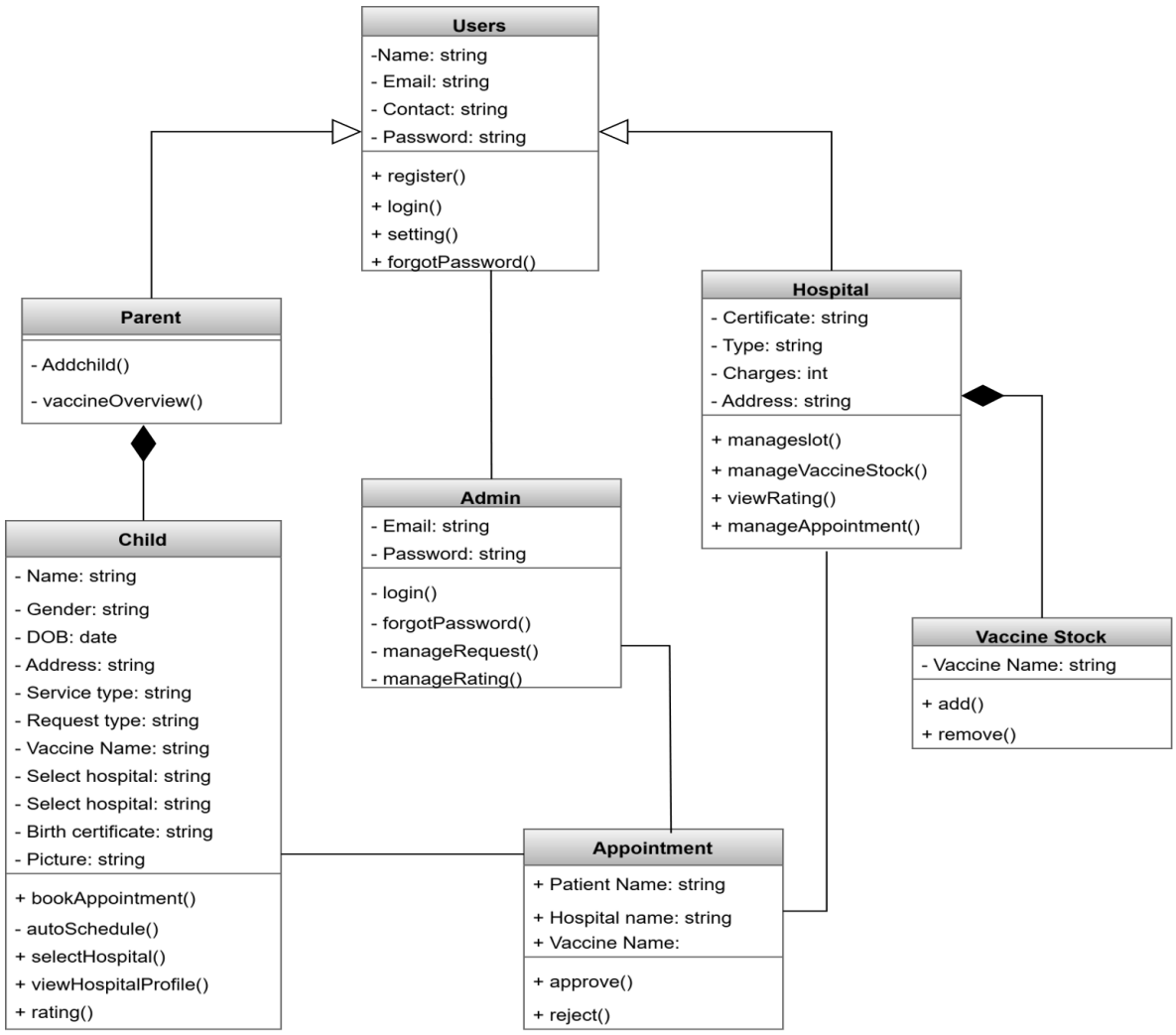


Figure 3-1 Class Diagram

3.2.3 Sequence diagram

The sequence diagram shows the interaction between components and objects with system. This diagram illustrates the interaction between the components of Kids Vaccination System. This diagram shows firstly the user send a request through UI. And then this information is sent to backend, Backend fetch the data from database, Database give response to Backend and Backend send the required to UI. This diagram shows step by step flow of each module or backend within the system.

User Page:

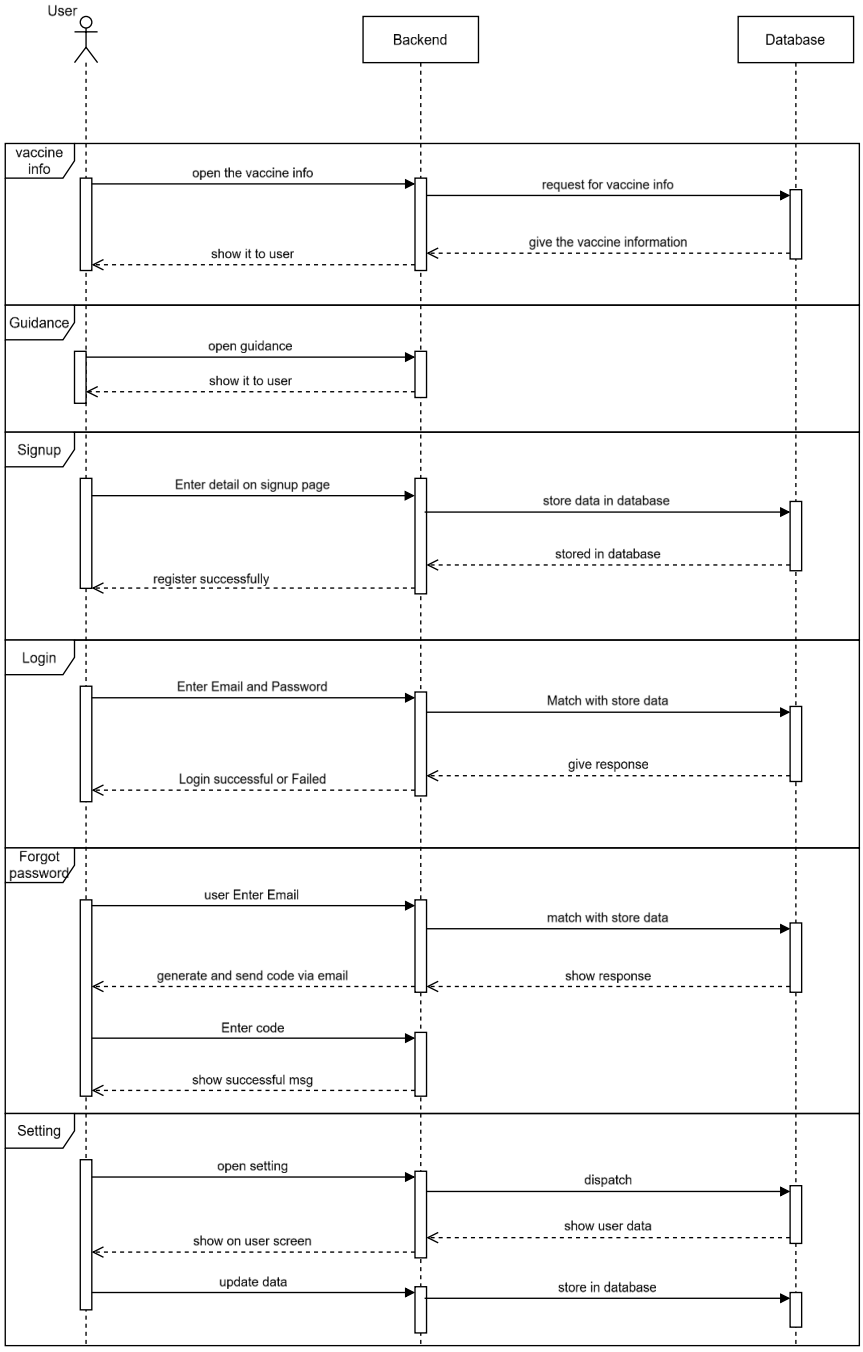


Figure 3-2 Sequence Diagram 1

Parent Panel:

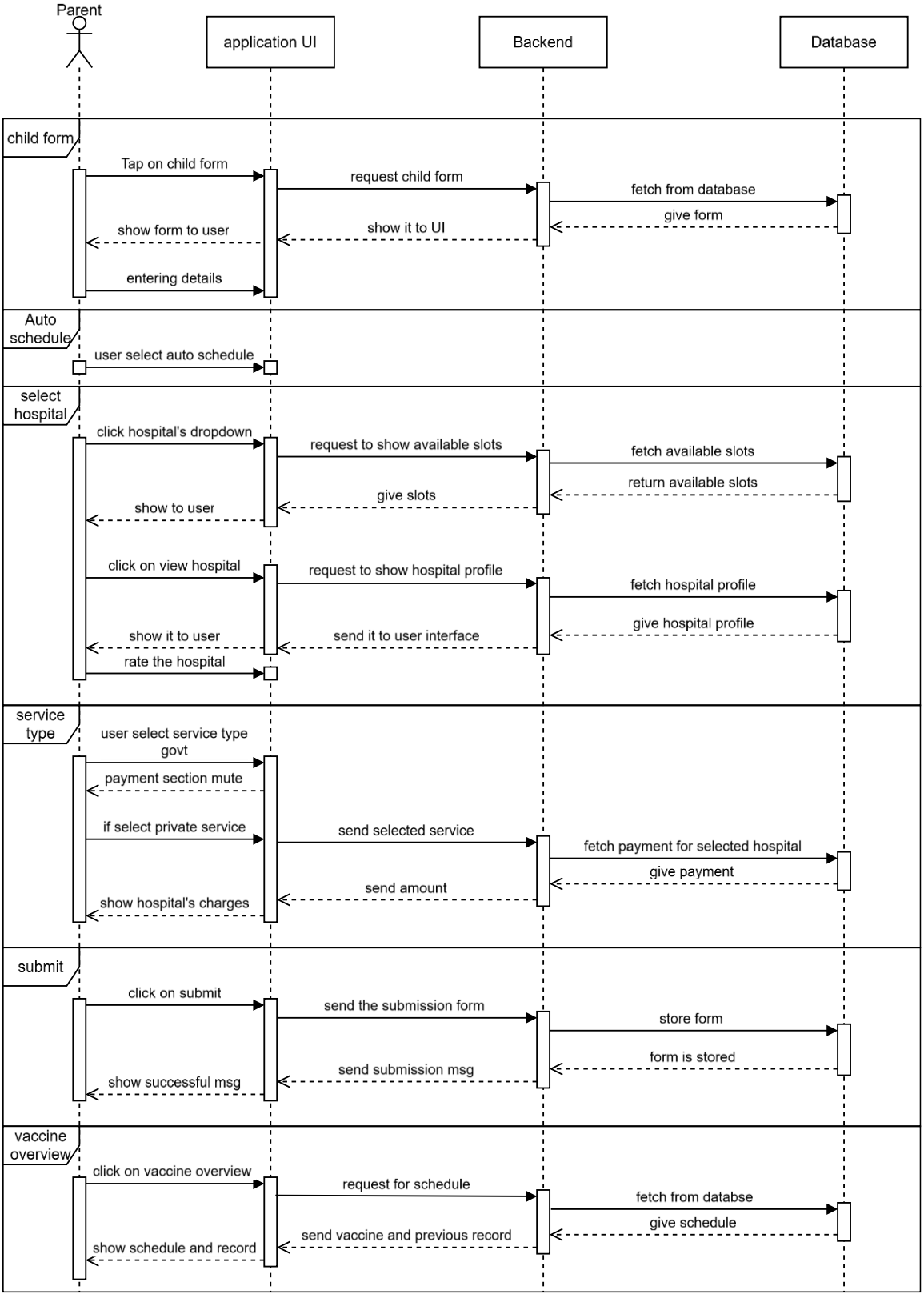


Figure 3-3 Sequence Diagram 2

Hospital Panel

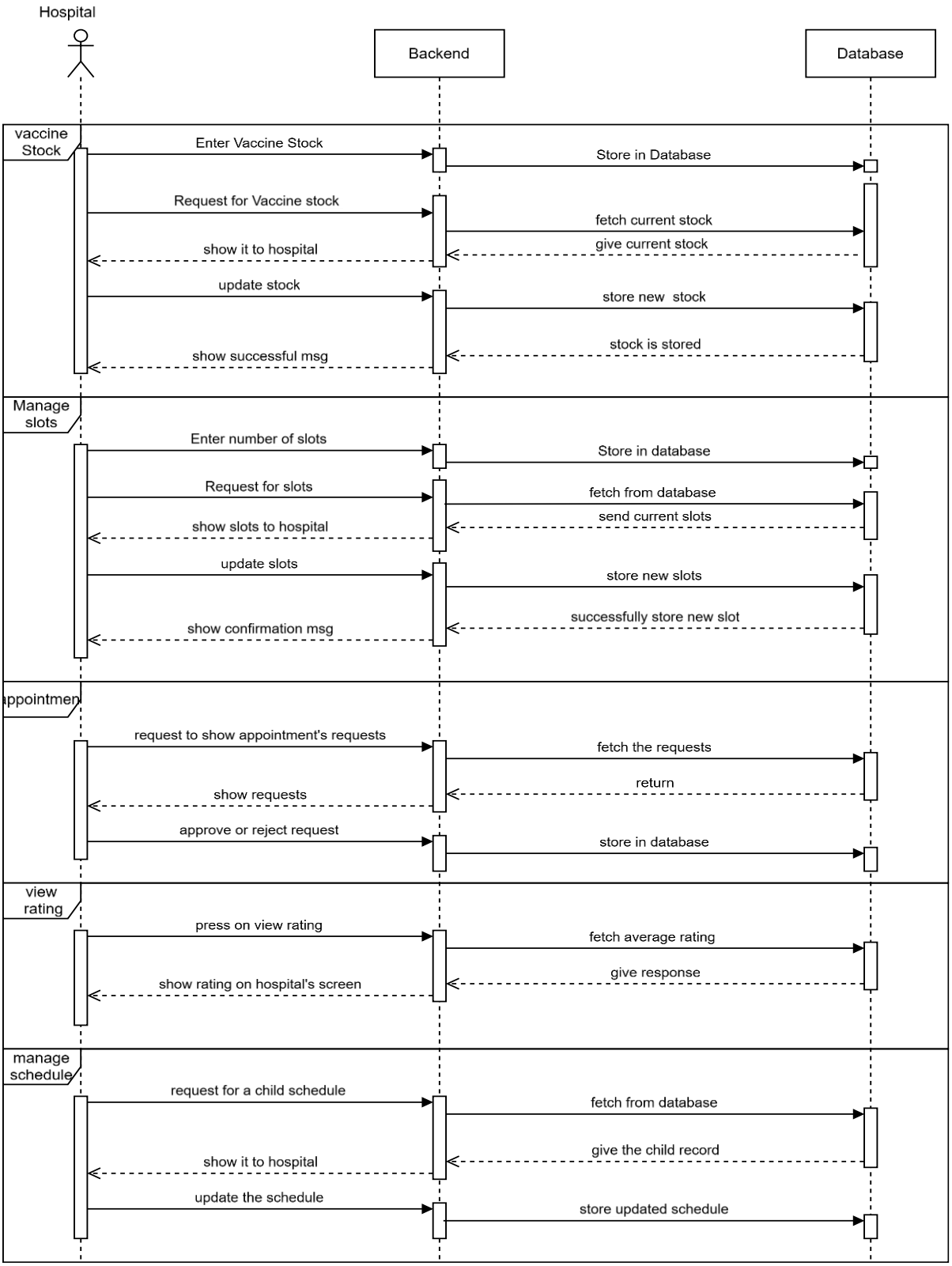


Figure 3-4 Sequence Diagram 3

Admin Panel:

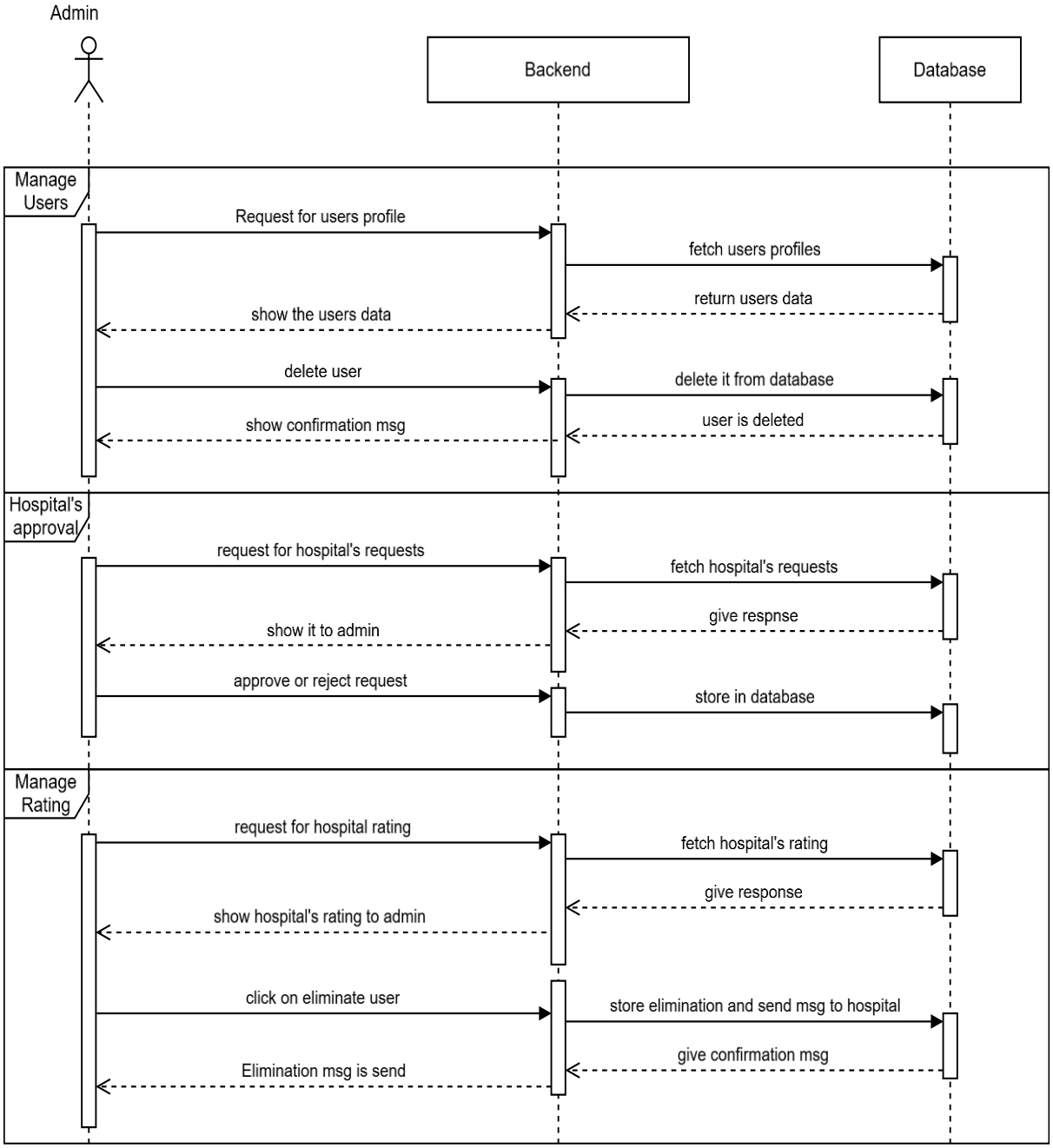


Figure 3-5 Sequence Diagram 4

3.1.4 State Transition Diagram

The state transition diagram represented the flow of different states for Kids Vaccination Booking website. The Diagram shows how the user performs an action by following different states and step by step flow of each state. For example, home screen shows different icons. User want to signup first they enter their credentials and then login into the system. Each state has a different function which is based on user action.

User Page:

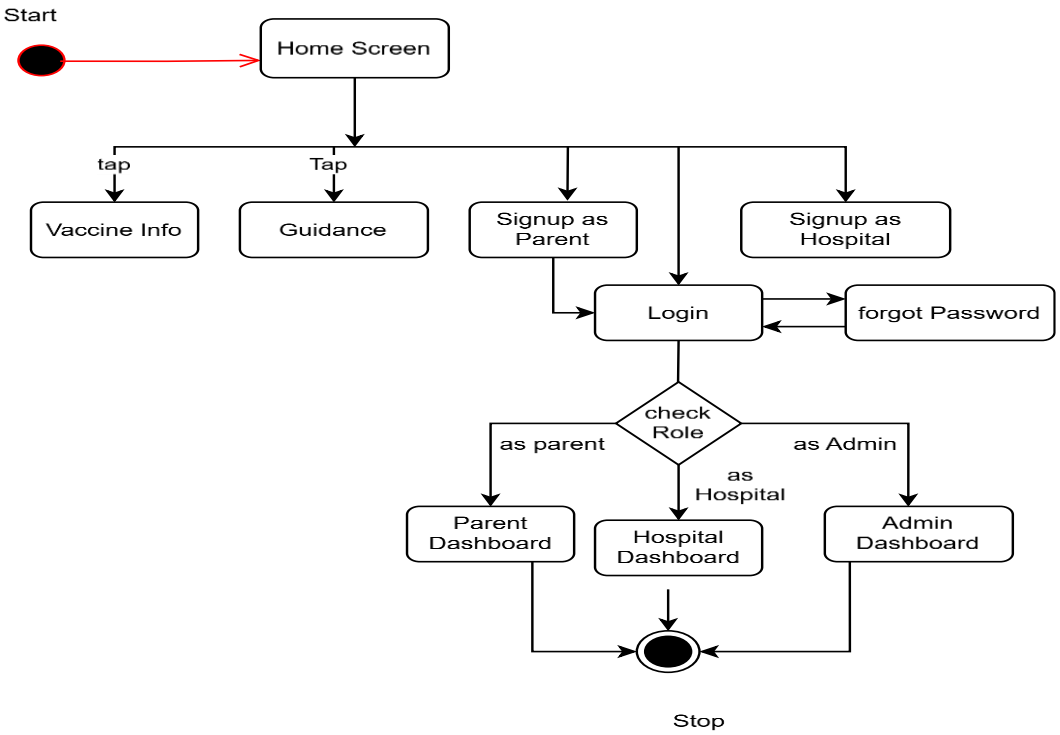


Figure 3-6 State Diagram 1

Parent Panel:

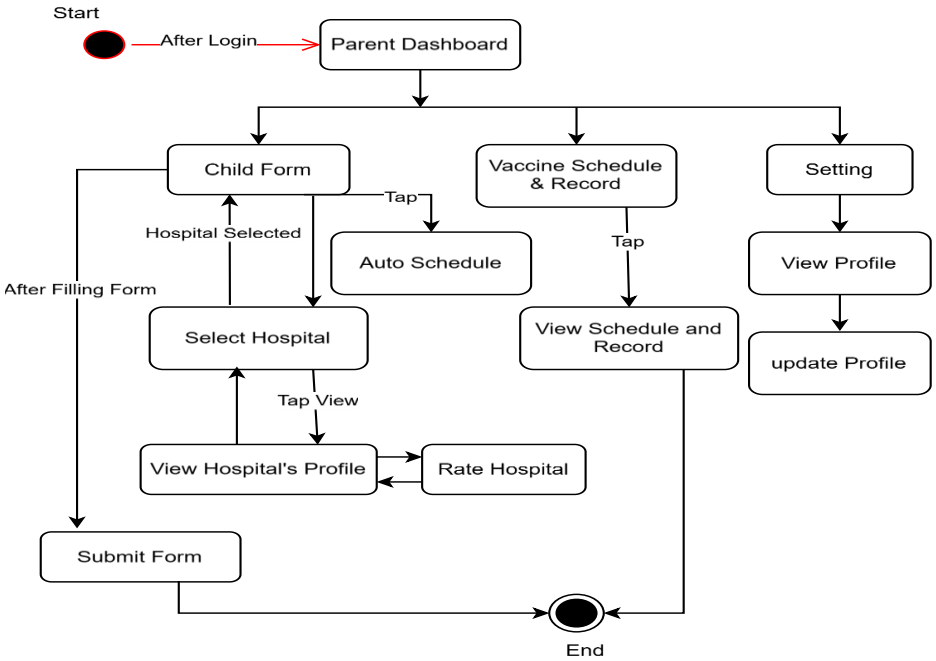


Figure 3-7 State Diagram 2

Hospital Panel:

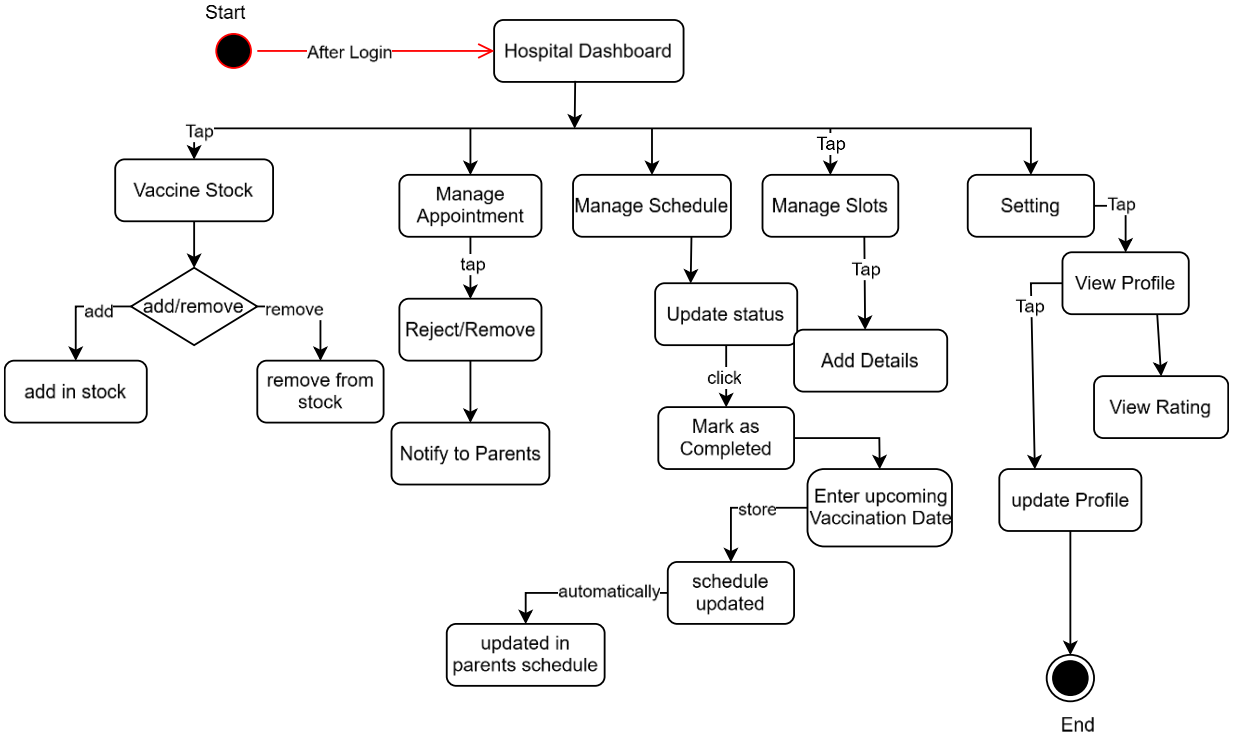


Figure 3-8 State Diagram 3

Admin Panel:

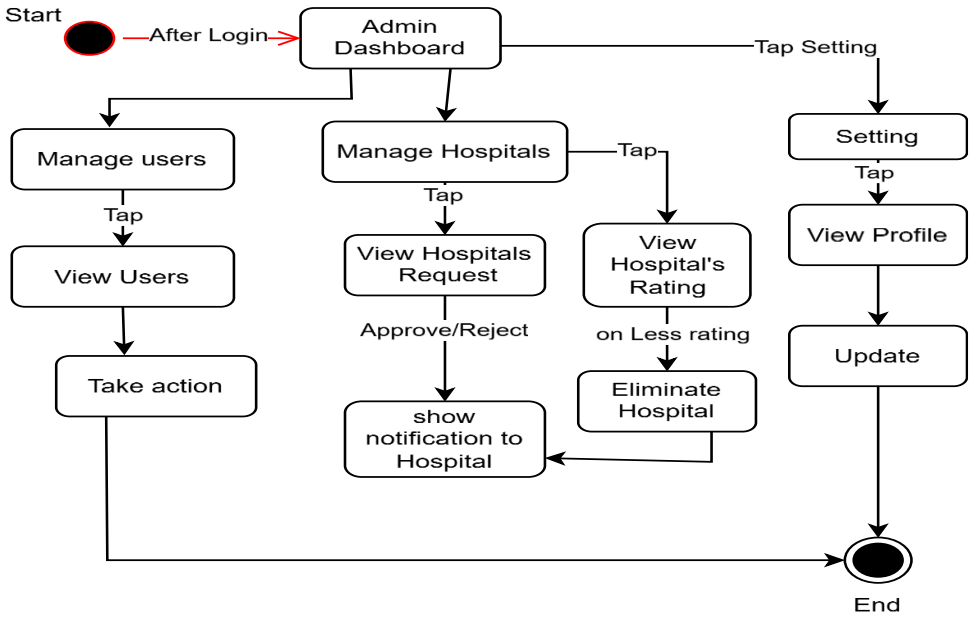


Figure 3-9 State Diagram 4

3.2 Architectural Design

3.2.1 Component Diagram

Component diagram illustrates the major components of kid's vaccination system and their relationship. This diagram shows UI for users to interacting with system and backend to perform respective task and a database for storing data.

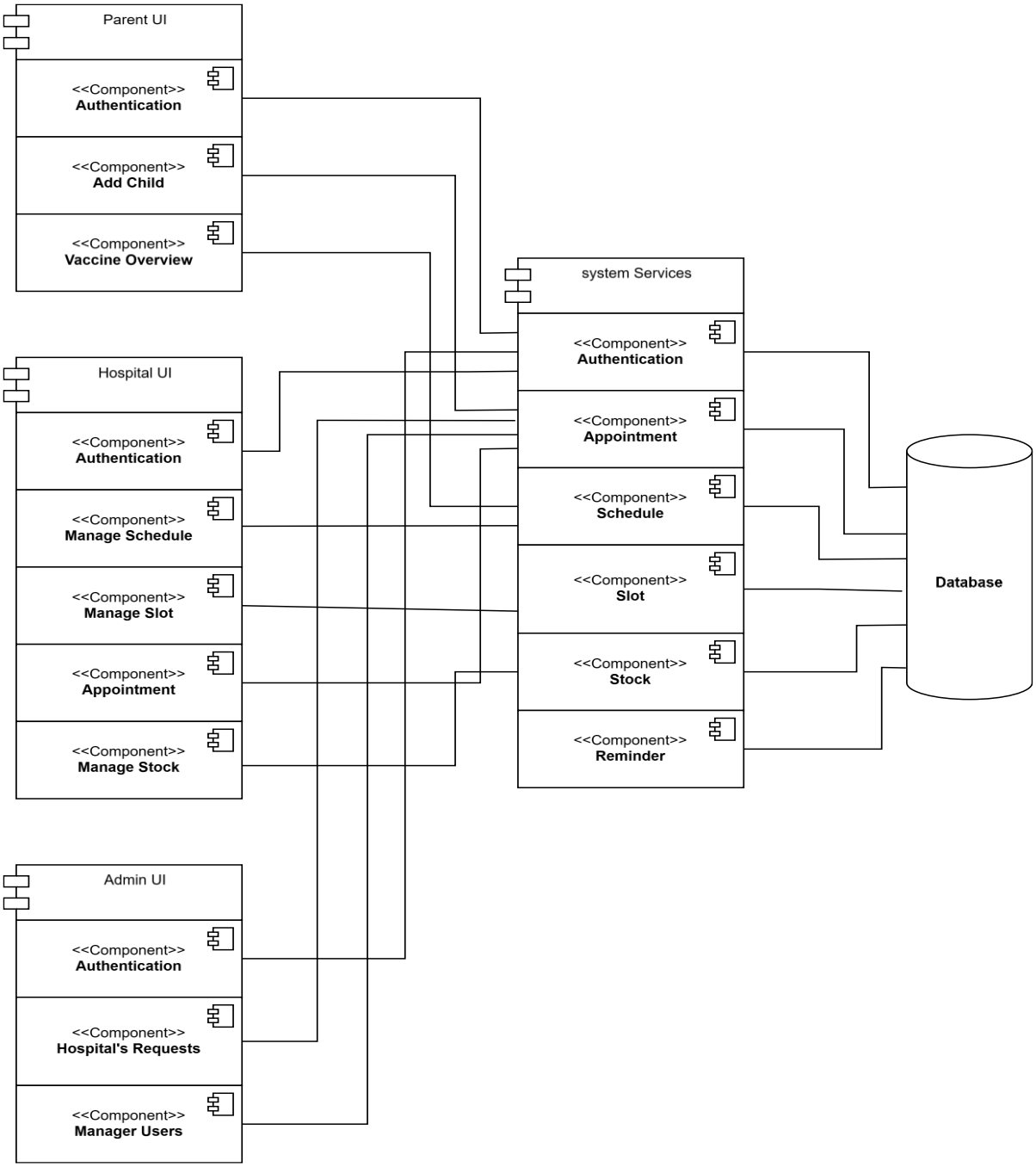


Figure 3-10 Component Diagram

3.2.2 N-Tier Architecture

Presentation tier represents the Frontend users, hospitals, admin and connect all the Frontend requests with backend. Application tier fetches the data from database and provides thee requested services to users. It is also connected with external APIs. Data tier is responsible for storing the data in database and give responses when needed.

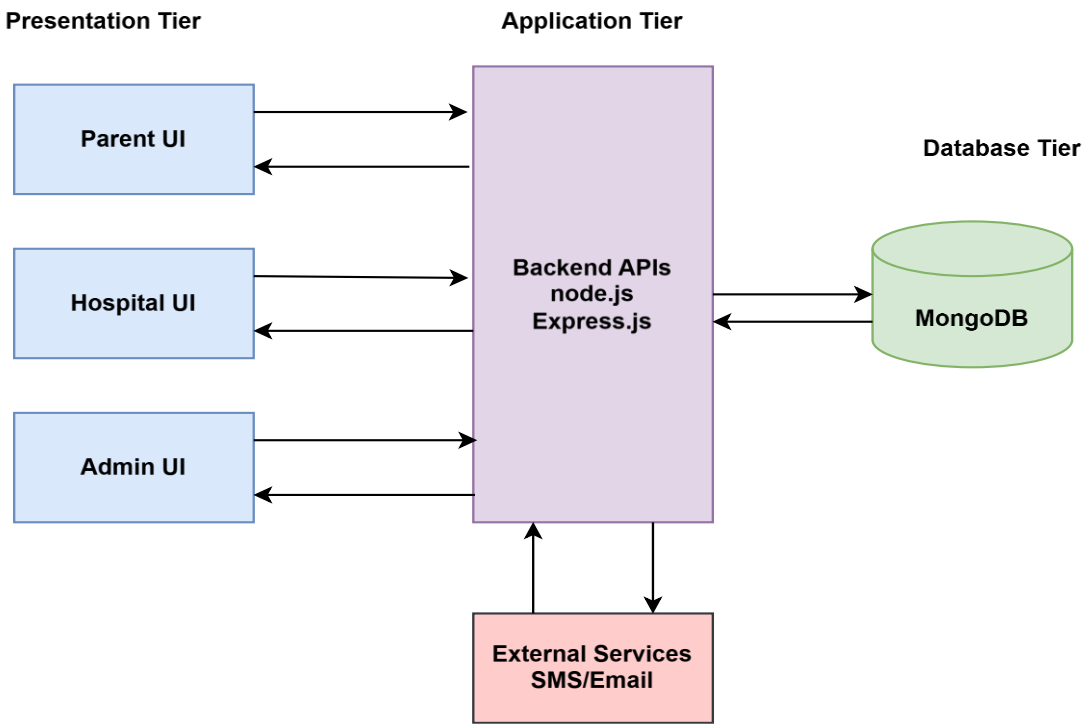


Figure 3-11 Architectural Diagram

3.3 Data Design

The information domain of kid's vaccination booking are parents, hospitals and admins, Appointments, vaccine stock and child.

These information domains are transformed into data structures into collections.

Parent Collection

- Parent ID
- Name
- E-Mail
- Contact
- Password
- Role

Children Collection:

- Child ID

- Guardian
- Parent ID (reference)
- Name
- Gender
- DOB
- Address
- Hospital
- AutoSchedule
- scheduleCompleted
- Parent
- status
- Request Type
- Service type
- Vaccine name
- Birth certificate
- Picture

Hospital collection:

- Hospital ID
- Name, E-mail
- Contact
- Password
- Type
- Visit type
- Address
- Charges (Home/Hospital)
- Certificate
- Role
- Stock
- Slot
- Rating

Admin Collection:

- Admin ID
- Name
- E-Mail
- Password
- Role

Notifications:

- ID
- Recipient
- Recipient role

- Title
- Message
- Read

Vaccines

- Vaccine ID
- name
- AgeInDays
- AgeLabel
- description

RELATIONSHIPS

User class is a super class of parent, hospitals and admin classes. Parent can add one or multiple children. A child can book one or many appointments and hospitals can approve or reject many appointments.
Admins can manage many hospitals and user's appointments.

ER Diagram:

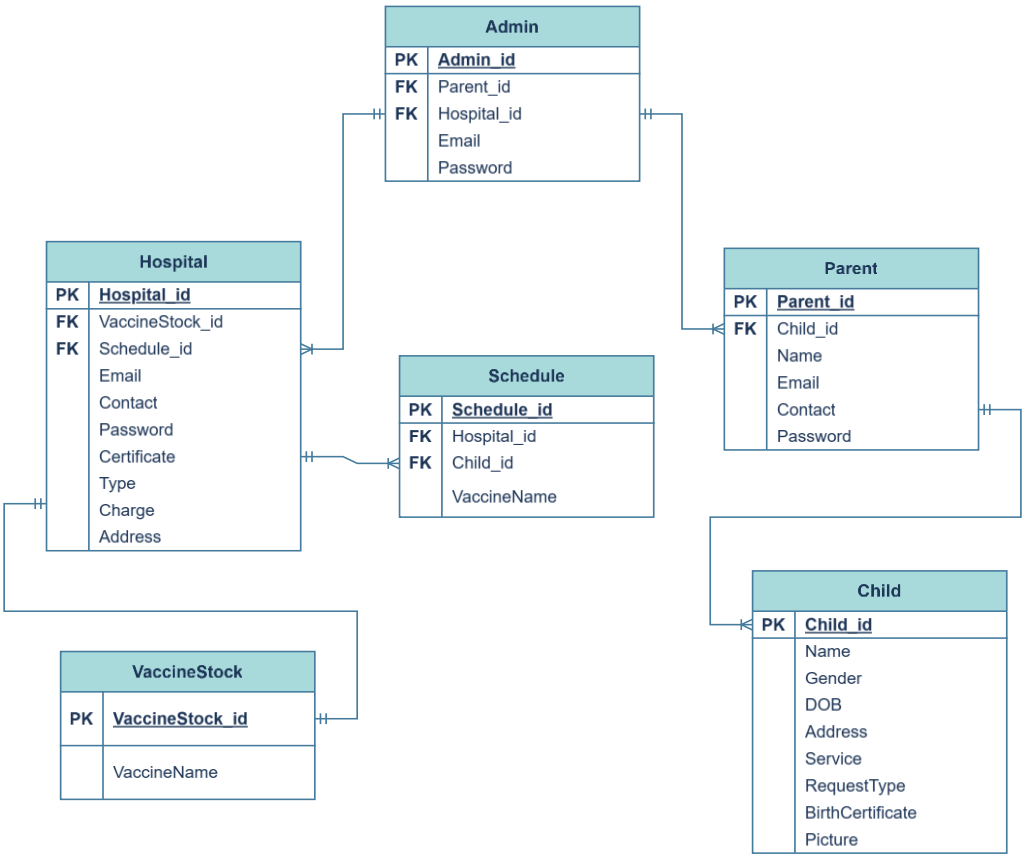


Figure 3-12 ER Diagram

3.3.1 Data Dictionary

Table 3-1: Data Dictionary

Terminology	Description
Admin	Class: Used for mange the parents, hospitals appointments and monitor rating.
Appointment	Class: Used for booking appointments from parents and managing by hospitals.
Auto schedule	Method: used to fetch the vaccines name according to DOB. Create schedule and display to the parents
Authentication	Method: Used to verify the user's (admins, hospitals, parents) credentials through database.
Child	Class: Used to store child's personal details.
Hospital	Class: Used for managing the appointments, schedule, slots and vaccine stock
Parent	Class: Used for booking child's appointment, view vaccine overview like records and schedule select and rate hospitals.
Rating	Method: Used to rate hospitals from users and display to the hospitals
Slots	Method: Slots are managed from hospitals and system display it to the parents.
User	Class: Users can be admin, hospitals and parents. It is used for register or login and interact with the system.
Vaccine Overview	Method: Hospital manage the schedule of upcoming vaccine of child and parents can view the records of completed vaccine and schedule of upcoming vaccines.

Structured Approach

Functions and Functions Parameters

There is a list of functions with their parameters that are used in the system.

Functions	Functions Parameters	Description
Calculate age	DOB (string): Childs Date of Birth.	Used to calculate the ages.

Generate auto-schedule	DOB (string): Childs Date of Birth.	Generate the schedule of vaccine according to the date of birth.
Available Slots	No. of slots(int): no of slots from hospitals. Time(string): Available time of hospitals.	Fetch number of slots from hospitals and display to the users.
Calculate rating	Rate(int): Rates from the users.	Calculate the number of rates.
Payment	Email(string): Parent's Email. Phone(string): Parent's phone no.	Send automatically reminders to the users through SMS/E-mail.
Send Reminders	Charges of hospitals.	Automatically show payment charges on private service selection.

Object- Oriented Approach:

For object-oriented approach, here is a list of objects with their attributes and methods with their parameters.

Objects	Attributes
Parent	• Name (string): Represent the name of parents. • E-mail(string): Represent the email passwords. • Contact no:(string): Represent the contact number of users. Password(string): Gives the password for security.
Hospital	• Name(string): Represent the name of hospitals. • Email(string): Represent the email of hospitals. • Contact(string): Represent the contact number. • Certificate(string): Save the URL of images. • Type(string): Represent the type of hospital either govt or private. • Charges(string): Represent the fee of hospitals. • Address(string): Represent the address of hospitals. • Password(string): Represent the password for security.
Admin	• E-mail(string): Represent the email of admins that is stored in database. • Password(string): Represent the password that is stored in database.
Child	• Name(string): Represent the name of children.

	• Gender(string): Represent the gender of hospitals • DOB (string): Represent the date of birth of child for calculating age. • Vaccine name(string): Represent the name of vaccine. • Service type (Boolean): Select the service of the hospital. • Birth Certificate(string): By storing the address of image and represent the image. • Picture(string): By storing the address of image and represent the image.
Appointment	• Child name(string): Represent the name of child for which appointment is book • Hospital name(string): Represent the name of hospital name. • Vaccine name(string): Represent the name of vaccine.
Vaccine stock	• Vaccine name(string): Represent the name of vaccine.

Methods with parameters:

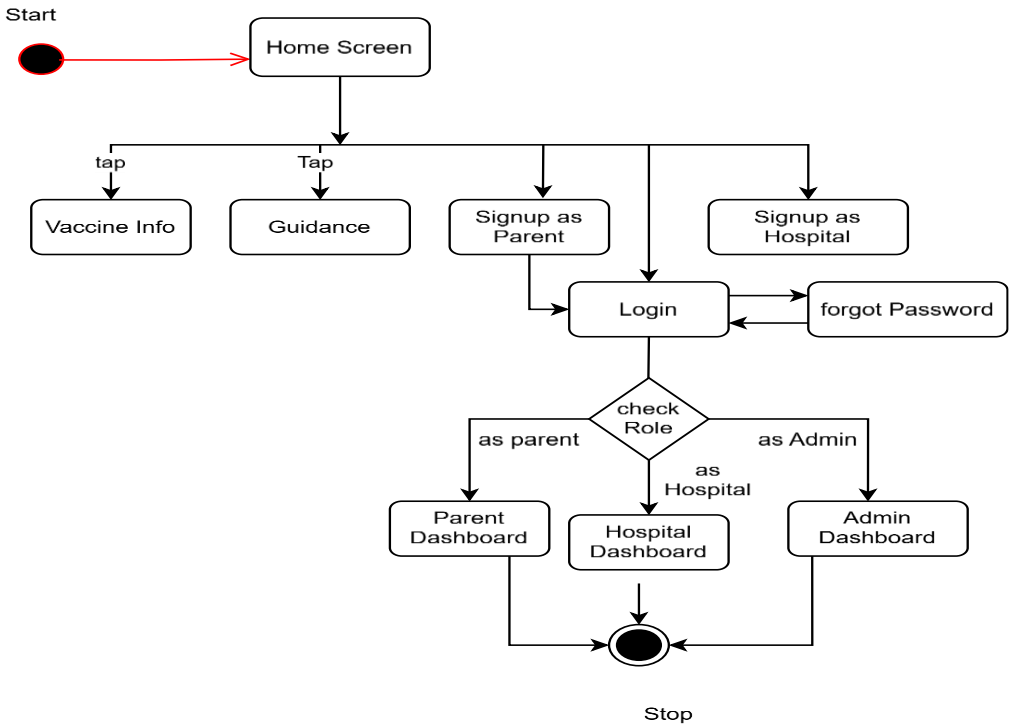
Objects	Methods with Parameters
Parent	• **Register** (name, E-mail, Password, Contact): For the registration of parents before booking. • **Login** (E-mail, Password): For the authentication purposes. • **Forgot password** (Email): For resetting the passwords. • **Add child** (name, gender, DOB, vaccine name, service type, hospital selection, birth certificate, picture, rate, payment): Filling the child form for booking. • **View Vaccine overview** (child's name, Hospital's name, vaccine name, date): View the upcoming vaccines schedule and the previous records.
Hospital	• **Register** (name, E-mail, password, charges, type, contact): For the registration of hospitals. • **Login** (E-mail, Password): For the authentication purposes. • **Forgot password** (E-mail): For resetting the passwords. • **Manage slots** (no. of slots, time): Hospital enters their time and this display to users. • **Manage schedule** (child name, date, status, DOB, picture, certificate): Hospitals mark the completed vaccine of child and assign the date of upcoming vaccine).

Appointment	• **Approve/Reject:(**child's name, vaccine name, hospital name, DOB etc.): Hospitals can approve or reject the appointments of users.
Vaccine	• **Add/Remove** (vaccine name): Hospitals can add or remove the vaccine details.
Admin	• **Login** (E-mail, Password) For the authentication purposes. • **Forgot password** (E-mail): For resetting the passwords. • **Manage Hospital's Approval** (Hospital name, certificate, charges, type, email, address): Admins can approve or reject the hospitals request. • **Manage user** (child details): Admin manage the user's appointments. • **Manage Rating** (hospital name, rate): Admins can view the rate from users and take actions against hospitals.

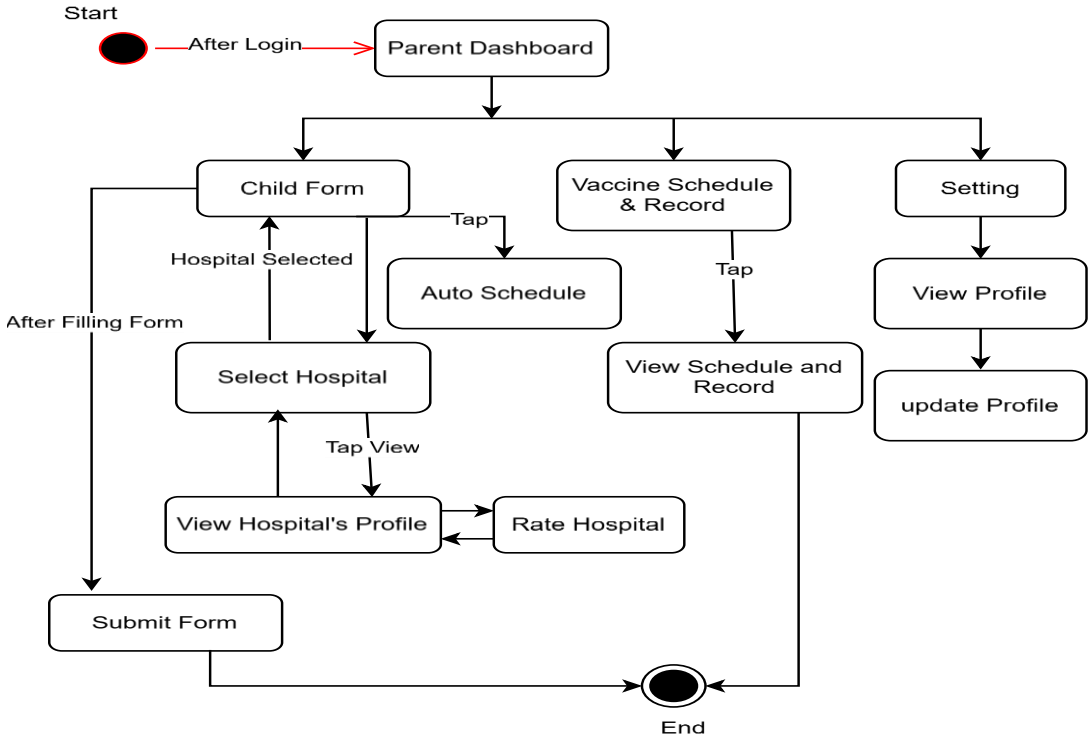
3.4 Behavioral Model

State Diagram

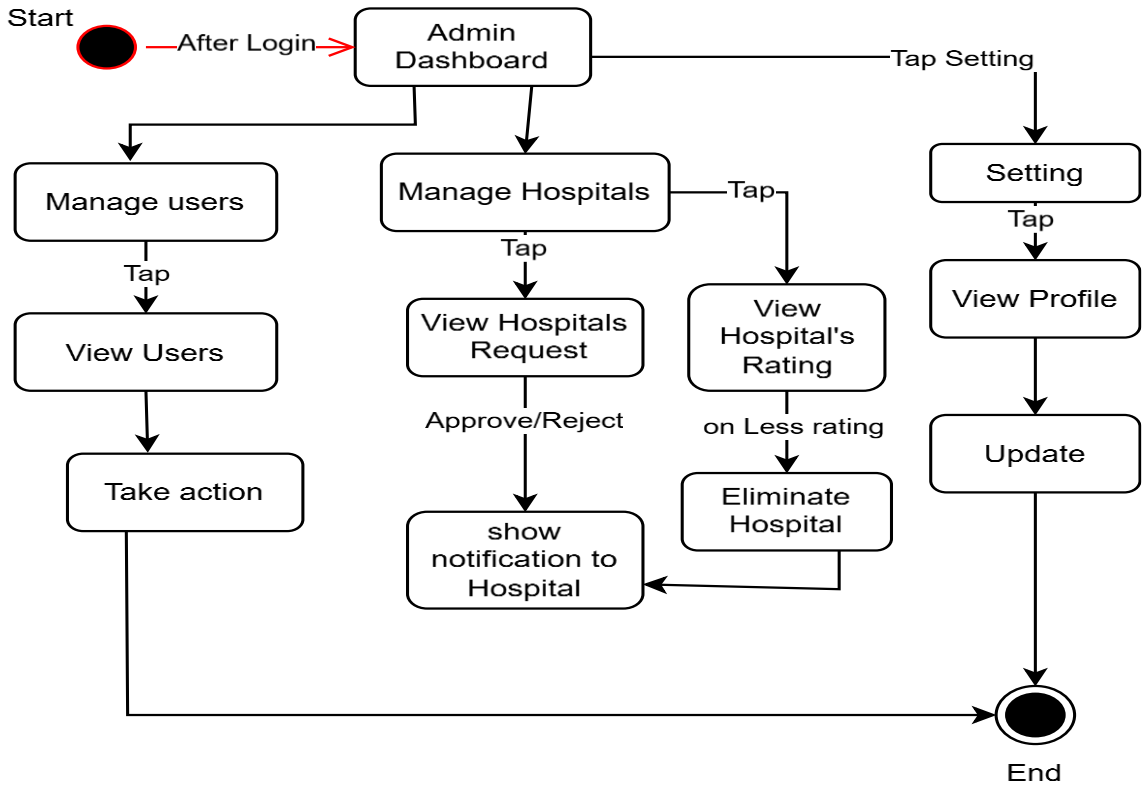
User Panel:



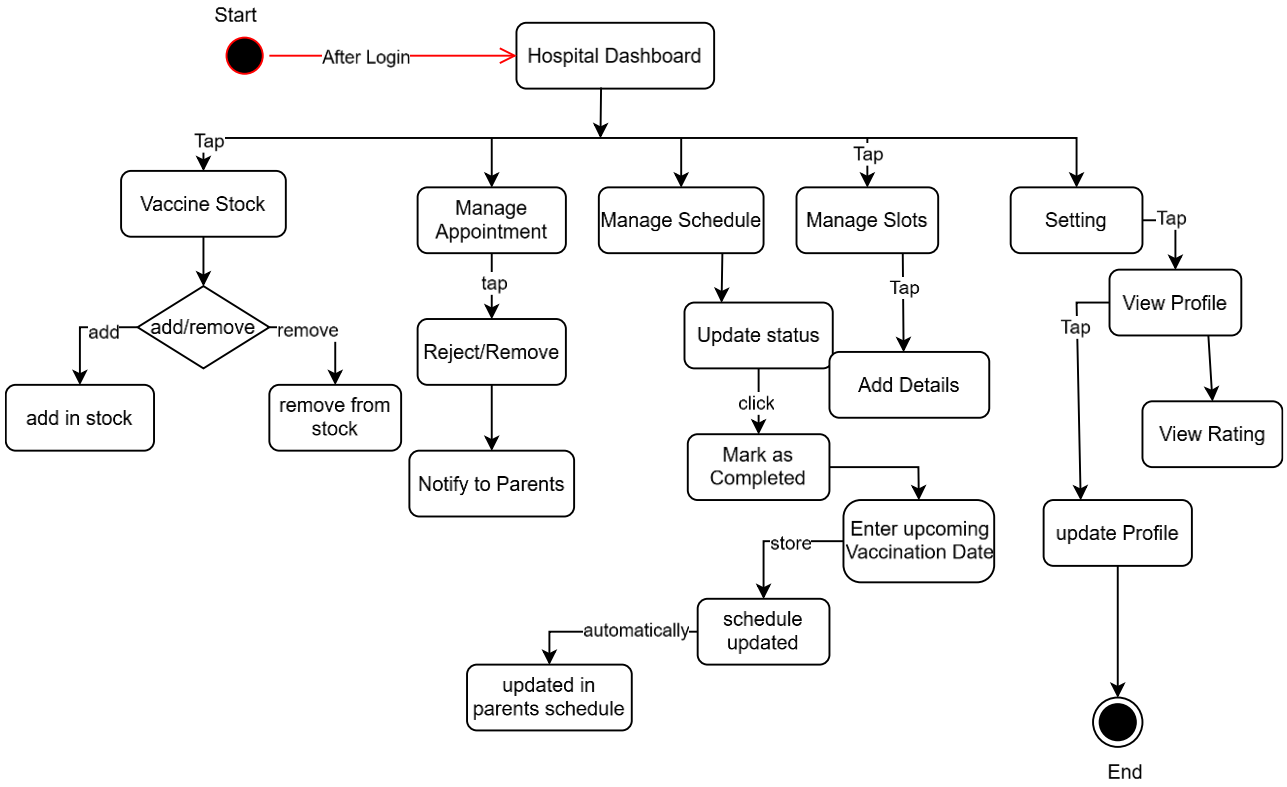
Parent Panel:



Admin Panel:



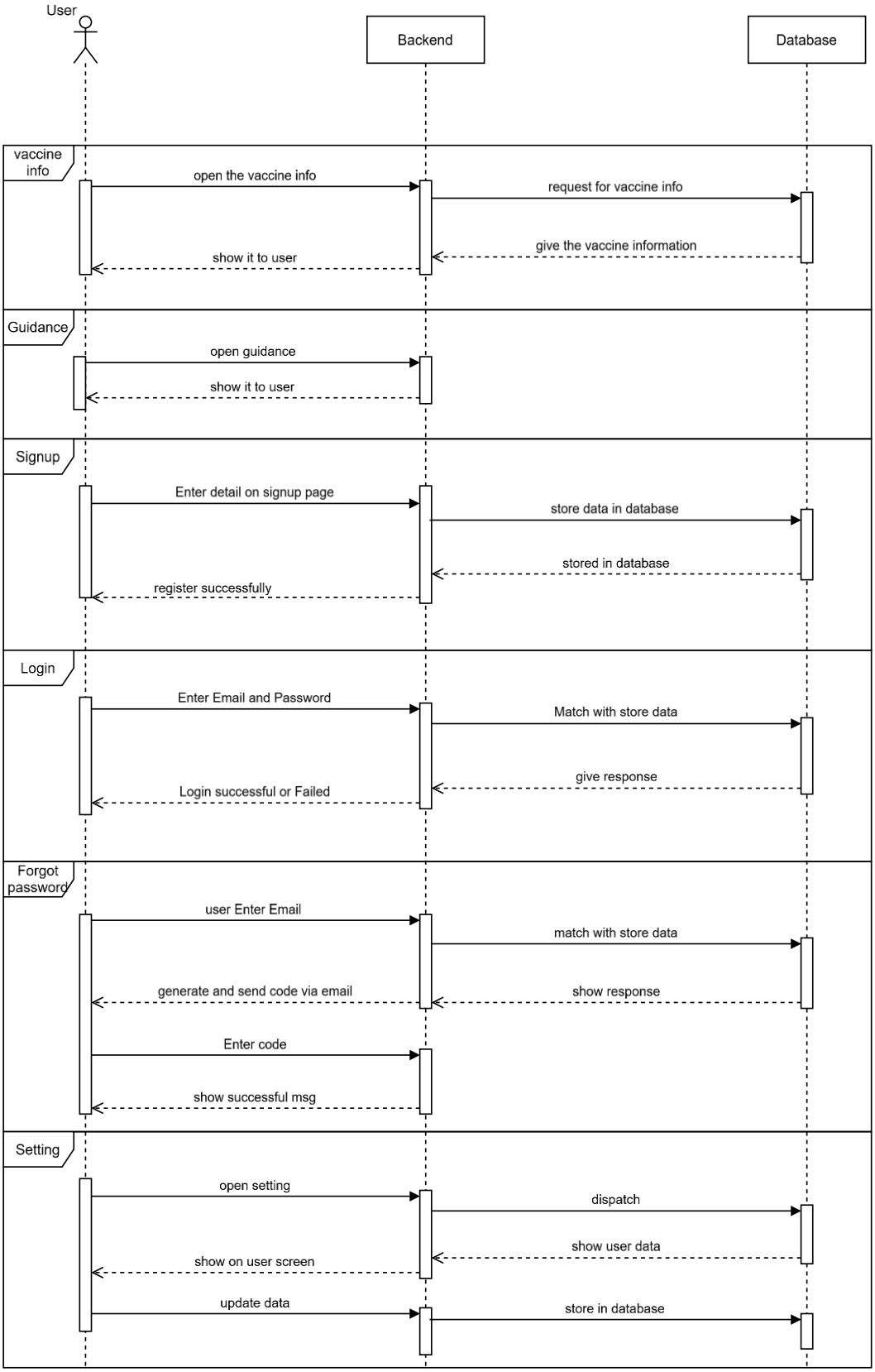
Hospital Panel:



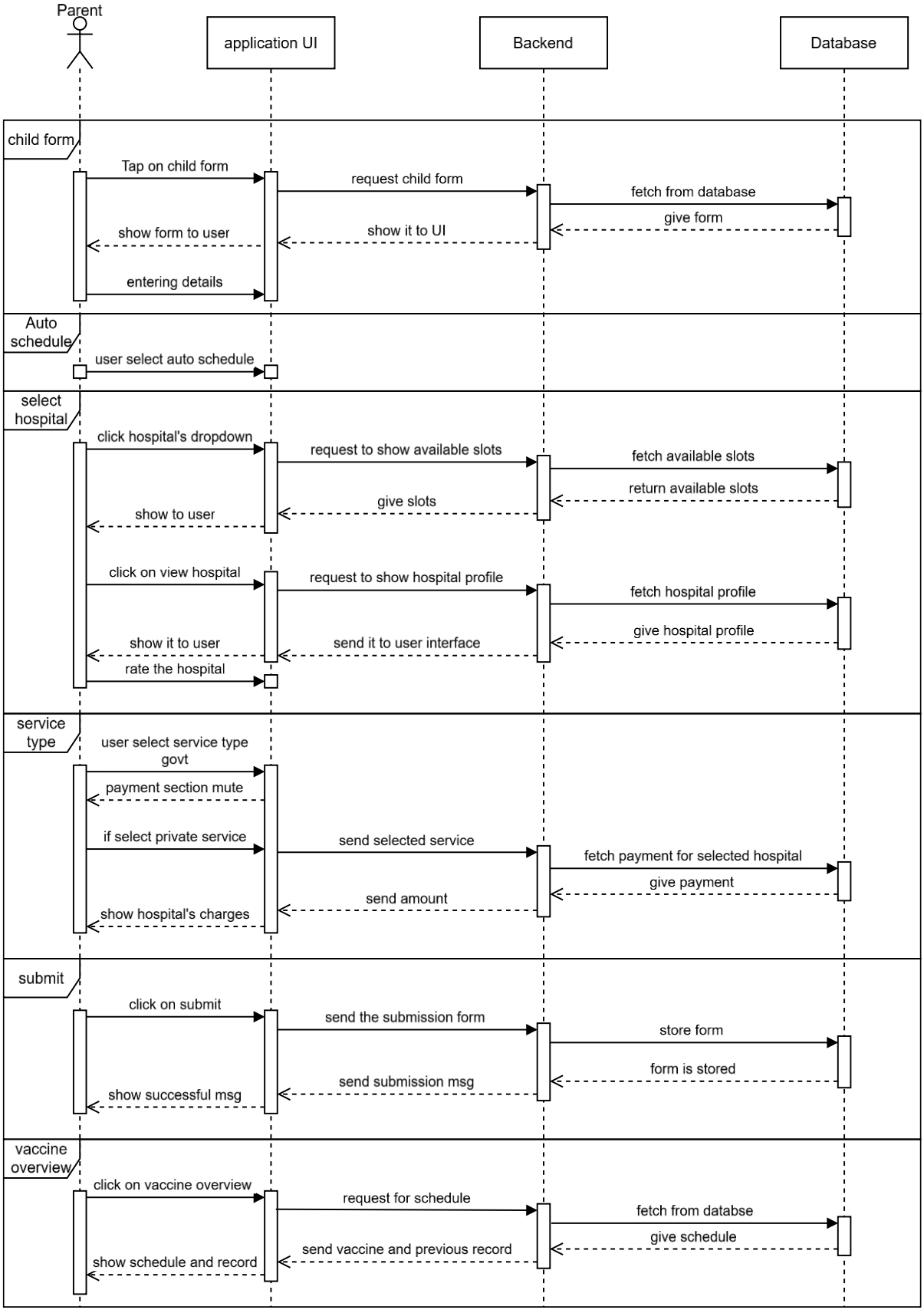
Sequence Diagram

The sequence diagram shows the interaction between components and objects with system. This diagram illustrates the interaction between the components of Kids Vaccination System. This diagram shows firstly the user send a request through UI. And then this information is sent to backend, Backend fetch the data from database, Database give response to Backend and Backend send the required to UI. This diagram shows step by step flow of each module or backend within the system.

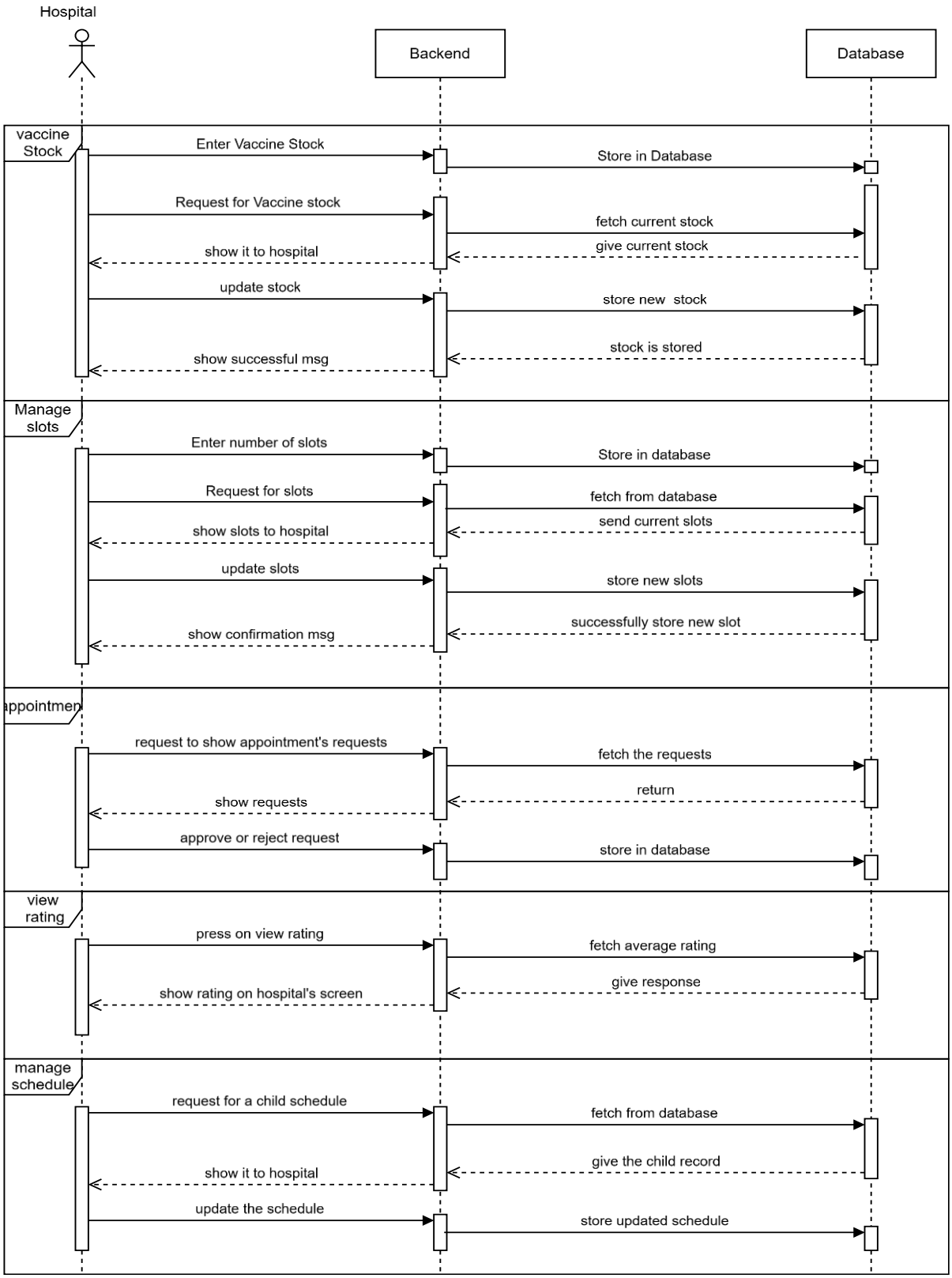
User Panel:


User
Backend
Database
vaccine info
open the vaccine info
request for vaccine info
give the vaccine information
show it to user
Guidance
open guidance
show it to user
Signup
Enter detail on signup page
store data in database
stored in database
register successfully
Login
Enter Email and Password
Match with store data
give response
Login successful or Failed
Forgot password
user Enter Email
match with store data
generate and send code via email
show response
Enter code
show successful msg
Setting
open setting
dispatch
show user data
show on user screen
update data
store in database

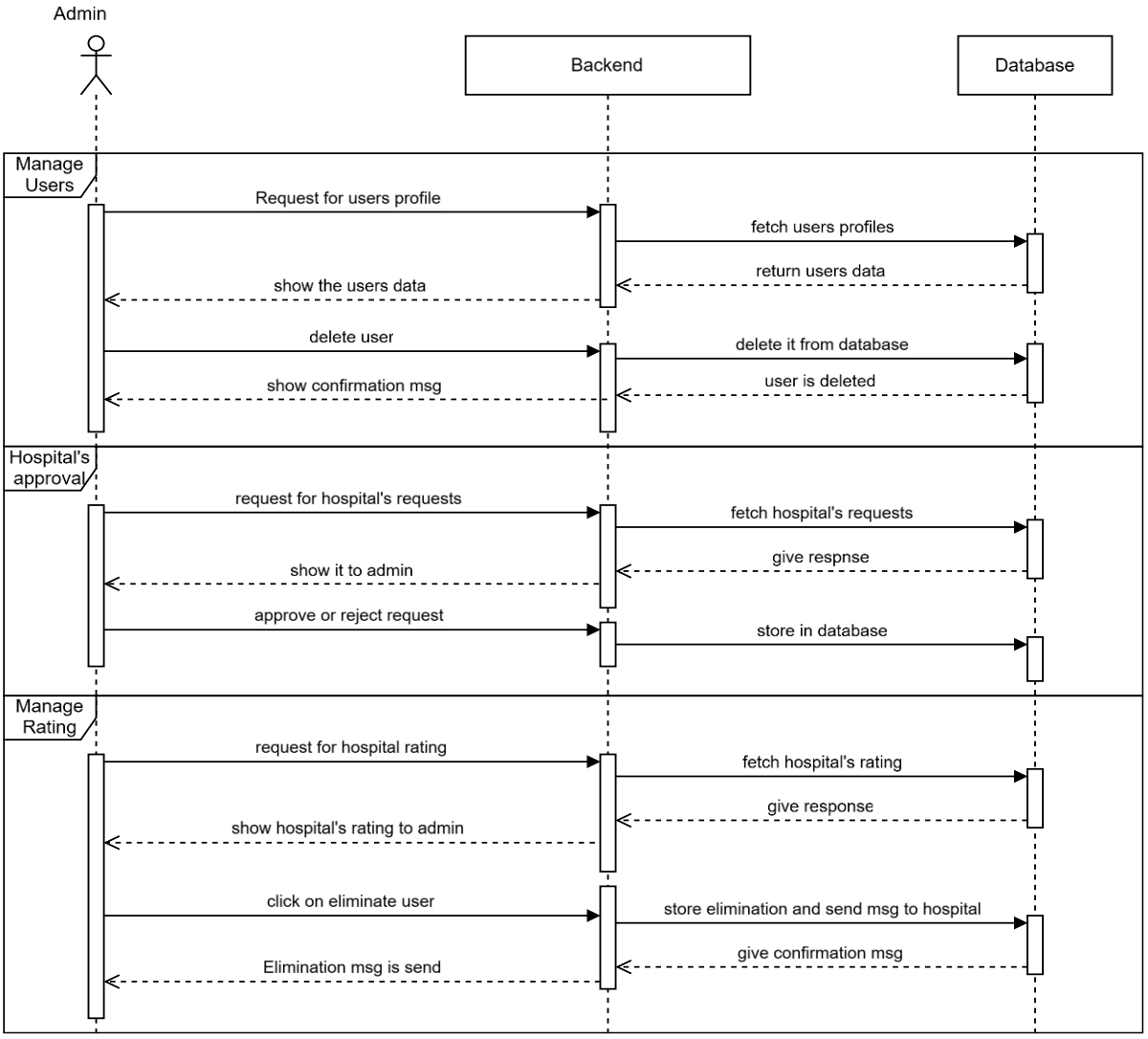
Parent Panel:



Hospital Panel:



Admin Panel:



3.4 Design Decisions

For the project N-tier architecture is used. Because kid's vaccination booking is divided into the layers. And this diagram is suitable for representing layers by maintaining the separation between layers. For storage MongoDB is used which us NoSQL based. It is easy to use and implement. This database is denormalized because it does not require a strict normalization like RDBMS. It is object-oriented approach because it involves classes and objects with their attributes and methods. A step-by step business logic and efficient algorithm is used making the system smooth and organized.

3.5 User Interface Design

3.5.1 Screen Images

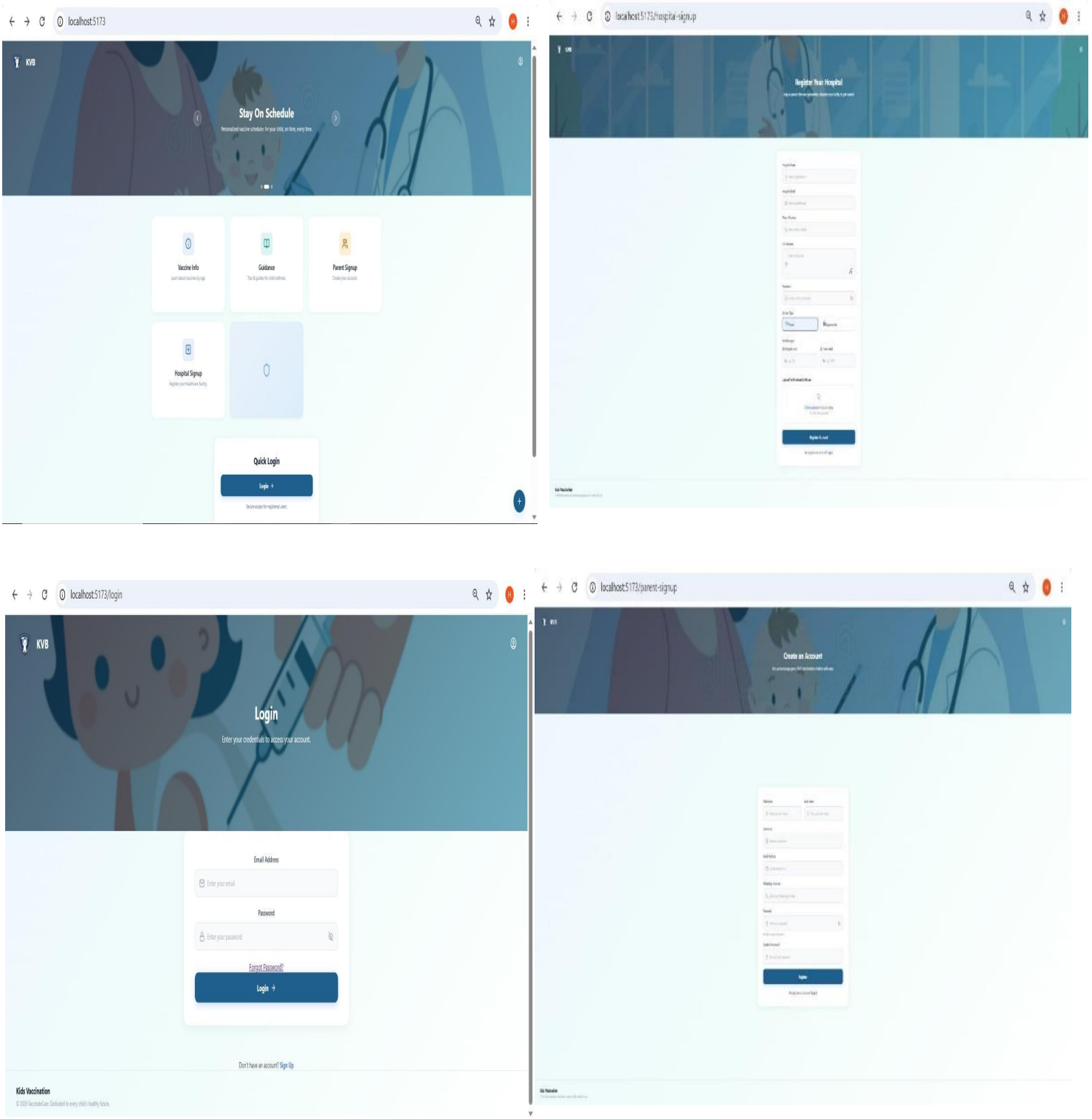


Figure 3-13 Screen Images

3.5.2 Screen Objects and Actions

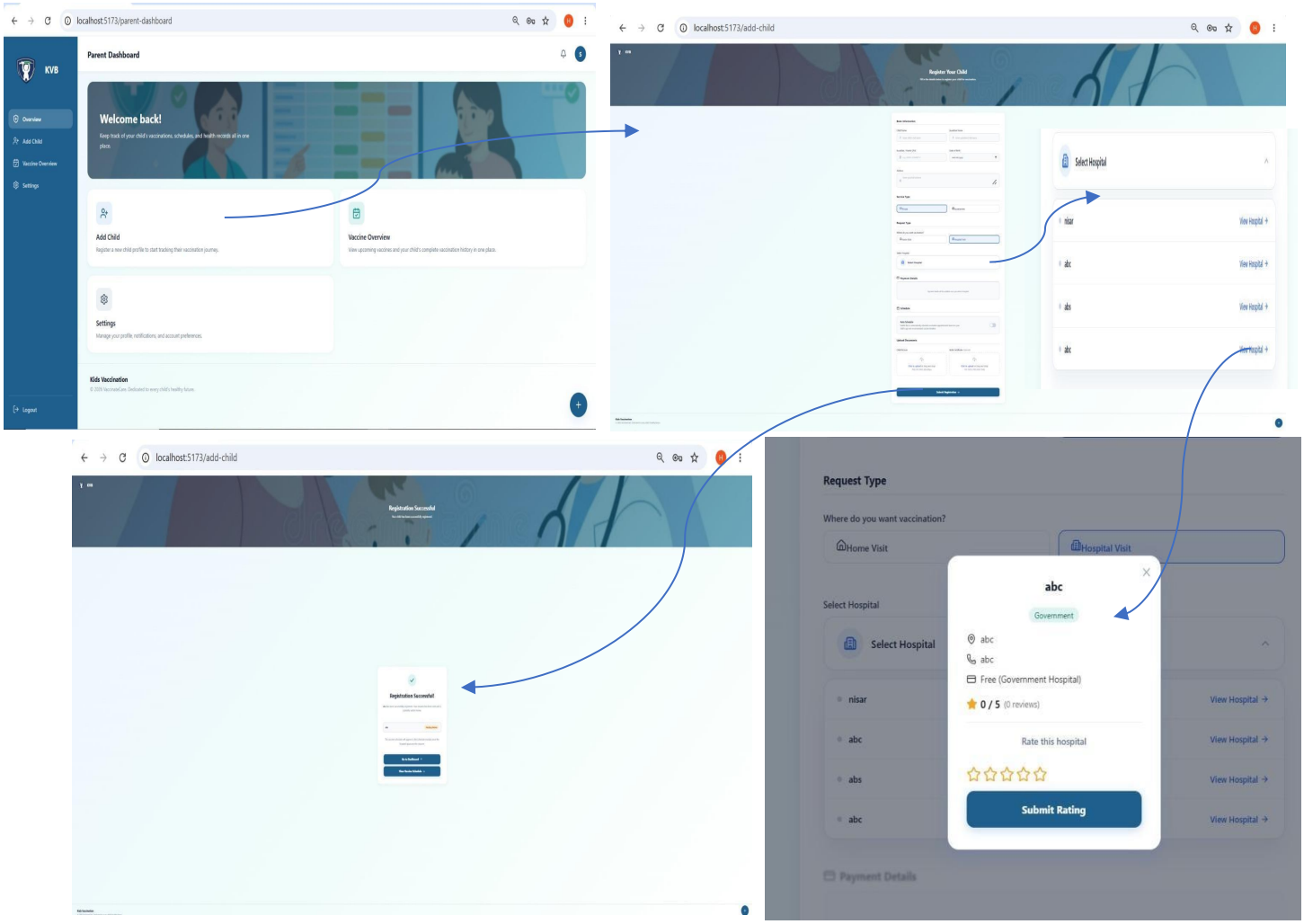


Figure 3-14 Screen Objects & Action

3.5 Summary

This section presents the comprehensive review of the system design, considerations and architecture. It consists of state, class, sequence, use-case and component diagram.

Chapter 4

Implementation

Implementation

This chapter discuss the implementation detail of Kids Vaccination Booking Platform. The main Component of the system are described in pseudo code form. The functionality of key modules is given to understand the overall system functionality.

4.1 Algorithms

Algorithm 1-Login and Authentication

Table 4-1 Algorithm 1

Section	Details
Algorithm	Login
Input	Email, Password, role ({Parent, Hospital, Admin})
Output	JWT Auth Token OR Authentication Error Message
Pseudo Code	function Login(formData) 1. onSubmit: 2. clear error 3. loading ← true 4. response ← POST /api/auth/login with { email, password } 5. data ← response.json() 6. if response not ok then 7. if data.pending == true then 8. pendingApproval ← true // hospital not yet approved 9. loading ← false 10. return 11. show error message (data.message or "Login failed") 12. loading ← false 13. return 14. store token in localStorage 15. store role in localStorage 16. if role == "hospital" then navigate → "/hospital-dashboard" 17. else if role == "admin" then navigate → "/admin-dashboard" 18. else navigate → "/parent-dashboard" 19. on network/server error: 20. show "Server error. Please try again later." 21. loading ← false

Algorithm 2-Add Child & Auto-Schedule Generation

Table 4-2 Algorithm 2

Section	Details
Algorithm	AddChild & Auto-Schedule Generation
Input	parentId, name, gender, DOB, serviceType, birthCertificate, picture
Output	Child Record + Auto-generated Vaccine Schedule
Pseudo Code	function AddChild(parentId, name, gender, DOB, serviceType, cert, pic) 1. if requestType == "hospital" AND hospital not selected then 2. return "Error: Please select a hospital" 3. ageInWeeks ← calculateAge(DOB) 4. schedule ← GenerateAutoSchedule(DOB, ageInWeeks) 5. if autoSchedule == true AND schedule is empty then 6. return "Error: Child not eligible — age beyond vaccination range" 7. child ← {parentId, name, gender, DOB, serviceType, 8. birthCertificate: cert, picture: pic} 9. DB.save(child) 10. return {child, schedule} function GenerateAutoSchedule(DOB, ageInWeeks) 1. vaccineTable ← DB.getVaccineTable() 2. schedule ← [] 3. for each entry in vaccineTable do 4. if ageInWeeks >= entry.minWeek AND ageInWeeks <= entry.maxWeek then 5. dueDate ← DOB + entry.dueWeeks 6. dose ← {vaccineName: entry.name, dueDate, status: "Pending"} 7. schedule.append(dose) 8. DB.saveSchedule(schedule) 9. return schedule function calculateAge(DOB) 1. today ← currentDate() 2. ageInDays ← today - DOB 3. return ageInDays / 7

Algorithm 3-Slot Management

Table 4-3 Algorithm 3

Section	Details
Algorithm	ManageSlots
Input	hospitalId, days, timeFrom, timeTo, totalSlots, type, slotId
Output	Updated Slots List (with Availability & Status: Open/Full) OR Error Message
Pseudo code	function ManageSlots(hospitalId) 1. slots ← GET /hospital/slots 2. if fails then show error 3. filteredSlots ← filterType == "All" ? slots : slots.filter(type == filterType) 4. display slots (day, time, total, booked, available, status) function AddSlot(days[], timeFrom, timeTo, totalSlots, type) 1. if any field empty then return 2. response ← POST /hospital/slots {days, timeFrom, timeTo, totalSlots, type} 3. if fails then show error 4. else update slots list, clear form function DeleteSlot(slotId) 1. response ← DELETE /hospital/slots/{slotId} 2. if fails then show error 3. else update slots list function EditSlotSave(slotId, day, timeFrom, timeTo, totalSlots, type) 1. response ← PUT /hospital/slots/{slotId} {day, timeFrom, timeTo, totalSlots, type} 2. if fails then show error 3. else update slots list, close modal function GetAvailability(slot) 1. return slot.totalSlots - slot.bookedSlots

Algorithm 4- View Appointment (Approve / Reject)

Table 4-4 Algorithm 4

Section	Details

Algorithm	View Appointments(parent) Approve/Reject
Input	hospitalId (via token), childId, action ({approve, reject, revert}), filterStatus
Output	Updated Appointment Status + Parent Notification
Pseudo Code	function ViewAppointments(hospitalId) 1. children ← GET /hospital/children-requests (with auth token) 2. if response not ok then 3. return "Error: Failed to load appointment requests" 4. store children list 5. filtered ← (filterStatus == "All") ? children : children.filter(c => c.status == filterStatus) 6. for each child in filtered do 7. display childName, serviceType, requestType, status, createdAt 8. on row click(childId) → 9. toggle expandedId == childId ? collapse : expand 10. if expanded then show childName, DOB, guardianName, guardianCnic, 11. parentPhone, address, serviceType, requestType, autoSchedule, 12. childPicture, birthCertificate 13. return {filtered, expandedDetails} function UpdateStatus(childId, action) 1. response ← PUT /hospital/children-requests/{childId}/{action} (with auth token) 2. result ← response.json() 3. if response not ok then 4. return "Error: " + (result.message or "Action failed") 5. update children list: replace matching child with result.child 6. return updatedChild function HandleApprove(childId) 1. return UpdateStatus(childId, "approve") function HandleReject(childId) 1. return UpdateStatus(childId, "reject") function HandleUndo(childId) 1. return UpdateStatus(childId, "revert")

Algorithm 5 – Manage Stock

Table 4-5 Algorithm 5

Section	Details
Algorithm	ManageStock (Add/Remove/Edit)
Input	hospitalId, vaccineName, quantity, itemId
Output	Updated Vaccine Stock or Error Message
Pseudo Code	function ManageStock(hospitalId) 1. stocks ← GET /hospital/stock 2. if fails then show error 3. display stocks (name, quantity, status) function HandleAdd(name, quantity) 1. if name or quantity empty then return 2. response ← POST /hospital/stock {name, quantity} 3. if fails then show error 4. else update stocks list function HandleDelete(itemId) 1. response ← DELETE /hospital/stock/{itemId} 2. if fails then show error 3. else update stocks list function HandleEditSave(itemId, name, quantity) 1. response ← PUT /hospital/stock/{itemId} {name, quantity} 2. if fails then show error 3. else update stocks list, close modal function GetStockStatus(quantity) 1. return quantity <= 10 ? "Low Stock" : "Available"

Algorithm 6-Manage Vaccination Schedule

Table 4-6 Algorithm 6

Section	Details
Algorithm	ManageSchedule(Mark Complete & Update)
Input	hospitalId, childId, scheduleItemId, dueDate, action

Output	Updated Vaccine Schedule (Status: Completed/Pending, Due Date) OR Error Message
Pseudo Code	function ManageSchedule(hospitalId) 1. children ← GET /hospital/children-requests 2. if fails then show error 3. approvedChildren ← children.filter(status == "Approved" AND name matches search) 4. display each child's schedule (vaccine, age, status, due date) function SetVaccineDate(childId, scheduleItemId, dueDate) 1. if dueDate empty then show error "Select a date first" 2. response ← PUT /hospital/children-requests/{childId}/schedule/{scheduleItemId}/set-date {dueDate} 3. if fails then show error 4. else update child's schedule, clear date draft function MarkGiven(childId, scheduleItemId) 1. response ← PUT /hospital/children-requests/{childId}/schedule/{scheduleItemId}/mark-given 2. if fails then show error 3. else update child's schedule (status → "Completed") function UndoMark(childId, scheduleItemId) 1. response ← PUT /hospital/children-requests/{childId}/schedule/{scheduleItemId}/undo 2. if fails then show error 3. else update child's schedule (status → reverted)

Algorithm 7-Forgot Password

Table 4-7 Algorithm 7

Section	Details
Algorithm	ForgotPassword
Input	Enter E-mail
Output	Reset Password

Pseudo Code	function SendCode(email) 1. if email empty then show error "Enter your email" 2. response ← POST /auth/forgot-password {email} 3. if fails then show error (response.message OR "Something went wrong") 4. else step ← "otp" function VerifyOtp(email, otp) 1. response ← POST /auth/verify-otp {email, otp} 2. if fails then show error (response.message OR "Invalid or expired code") 3. else step ← "password" function ResetPassword(email, otp, newPassword, confirmPassword) 1. if newPassword.length < 8 then show error "Password must be at least 8 characters" 2. if newPassword ≠ confirmPassword then show error "Passwords do not match" 3. response ← POST /auth/reset-password {email, otp, password: newPassword} 4. if fails then show error (response.message OR "Something went wrong") 5. else step ← "done" 6. wait 2.5s → navigate to "/login" function GoBackToEmail() 1. step ← "email" 2. clear otp 3. clear error

Algorithm 8-Manage Hospital Ratings and Reviews

Table 4-8 Algorithm 8

Section	Details
Algorithm	ManageRatings
Input	adminId (via token), hospitalId, ratingId, search, filter
Output	Filtered Ratings List + Updated List (after remove/reject) OR Error Message

Pseudo Code	function RateHospital(parentId, hospitalId, rating) 1. verify authToken; assert role == "parent" 2. if rating < 1 OR rating > 5 then 3. return "Error: Rating must be between 1 and 5" 4. existingRating ← DB.find(parentId, hospitalId) 5. if existingRating != NULL then 6. existingRating.value ← rating 7. DB.update(existingRating) 8. else 9. DB.save({parentId, hospitalId, rating}) 10. avgRating ← DB.average(hospitalId, field: "rating") 11. hospital.averageRating ← avgRating 12. DB.update(hospital) 13. return avgRating function ModerateRating(adminId, hospitalId, action) 1. verify authToken; assert role == "admin" 2. hospital ← DB.find(hospitalId) 3. if action == "eliminate" then 4. hospital.status ← "Eliminated" 5. DB.update(hospital) 6. Notify.sendEmail(hospital.email, "Account eliminated") 7. return "Hospital eliminated" 8. else if action == "warn" then 9. Notify.sendEmail(hospital.email, "Warning: Low rating") 10. return "Warning sent"

Algorithm 9-Manage Hospitals and Parents (Admin)

Table 4-9 Algorithm 9

Section	Details
Algorithm	ManageHospital and ManageParents
Input	hospitalId, parentId, childId, search, filter
Output	Updated Hospital Status + Parent/Child Details + Updated Parent List OR Error Message
Pseudo Code	function ManageHospitals(adminId) 1. hospitals ← GET /admin/hospitals 2. filtered ← hospitals.filter(status == filter AND search match) 3. display filtered hospitals (name, type, fees, status)

	function ApproveHospital(hospitalId) 1. PUT /admin/approve-hospital/{hospitalId} 2. update status → "Approved" function RejectHospital(hospitalId) 1. DELETE /admin/reject-hospital/{hospitalId} 2. remove hospital from list function ManageParents(adminId) 1. parents ← GET /admin/parents-with-children 2. filtered ← parents.filter(search match) 3. display filtered parents (name, email, children count) function ViewChild(childId) 1. display child DOB, hospital, birthCertificate 2. display vaccination records (vaccine, dateGiven, status) function DeleteParent(parentId) 1. DELETE /admin/parents/{parentId} 2. remove parent from list

4.2 External APIs/SDKs

API / SDK Name & Version	Description	Purpose of Usage	End Point / Function / Classes used
Lifetimesms (v5.10.6)	It is used to send automatically notification through mobile phone SMS.	Send notification about appointment confirmation and rejection and vaccination reminders.	sendSMS ()
Nodemailer (v8.0.6)	A Nodemailer module used for sending emails through SMTP.	To send email notifications for appointment confirmation and rejection and vaccination reminders.	transporter. sendMail ()

bcrypt.js (v5.0.0)	A library for hashing passwords before saving in the database so real password are not saved in plain text.	To securely hash parent, hospital, and admin passwords before storage, and to compare passwords at login.	bcrypt. hash (), bcryptcompare () Used in: controllers/authController.js
JSON Web Token – jsonwebtoken (v9.0.2)	JSON Web Token used for authentication and authorization.	To authenticate users and verify their identity.	jwt. sign (), jwt.verify() Used in: middleware/authMiddleware.js, controllers/authController.js
Multer Storage Cloudinary (v4.x)	A tool that uploads files to Cloudinary.	To handle birth certificate and child profile picture uploads submitted in the child registration form.	CloudinaryStorage, multer()
Cloudinary	A cloud service used to store files and images.	A cloud service used to store files and images	cloudinary.config (), upload functions

4.3 Code Repository

Git Repository link: https://github.com/SH12345507/Kids-Vaccination-Booking

It's a public repository; the complete project code is there. It will be updated as we test the website and improve it.

4.4 Summary

This Chapter discussed the implementation detail of the Kids Vaccination System. The major algorithms were discussed with pseudo code, that cover the main functionality of the system Registration, Authentication, Booking, Vaccine stock and schedule Management. External APIs and SDKs were discussed which help in website development. A Git repository was also used to understand the flow of work and overall flow of the system.

Chapter 5

Testing And Evaluation

Introduction

This chapter focuses on the testing process of Kids vaccination booking system, to effective software verification and validation. The purpose of testing is to verify system accuracy ensure it meets users' needs and identify any critical issues. A structure testing approach is followed covering unit testing, Functional testing, integration testing and performance testing.

5.1 Unit Testing

Unit testing is performed to verify each individual module of the system works correctly and independently. Unit testing is applied to the component such as registration, login, generate auto-schedule, calculate rating, reminders and vaccine stock. It ensures each unit work accurately without depending on each other.

5.1.1 Test case1:

Table 5-1: Unit test Case 1

TC-ID	UT-1
Req.ID	R3
Title	Registration, Valid input
Description	Test the registration () method of the user module with valid name, email, password, and rate.
Objectives	Verify that a new user account is created and stored to the database when all the credentials are valid.
Pre-conditions	Database is running, no existing user with same e-mail.
Test steps	Call Registration () Enter valid data. Check user ID is not null.
Input	Name= "Hifza" Email= hllohifza2026@gmail.com password= "123456789" phone=03220000000 role= "parent"
Expected result	A new user account is created in database.
Actual Result	New user is created
Status	Pass

5.1.2 Test case2:

Table 5-2: Unit test case 2

TC-ID	UT-2
Req.ID	R5
Title	User login, Correct credentials.

Description	Test the login () with valid email and password.
Objectives	Verify that an existing user can successfully authenticate and can access the platform using valid credentials.
Pre-conditions	A user with email hllohifza2026@gmail.com already exists in database and currently logged out.
Test steps	Call login (e-mail, password). Check that a non-empty JWT token is returned
Input	Email= hllohifza2026@gmail.com password= "123456789"
Expected result	The user is successfully authenticated. No error is thrown.
Actual Result	User successfully logged in.
Status	Pass

5.1.3 Test case3:

Table 5-3: Unit test case 3

TC-ID	UT-3
Req.ID	R5
Title	User login, Incorrect password.
Description	Test the login () method with the incorrect password
Objectives	Ensure system returns an error when user enter invalid credentials
Pre-conditions	The email hllohifza2026@gmail.com exists in the database.
Test steps	Enter valid email and incorrect password. Call the login () function. Observe system response.
Input	Email= hllohifza2026@gmail.com password= "1234567"
Expected result	Return error. "Wrong password".
Actual Result	Pass
Status	Pass

5.1.4 Test case4:

Table 5-4: Unit test case 4

TC-ID	UT-4

Req.ID	R9
Title	Auto-schedule, Age calculation
Description	Test the calculate age and generate auto-schedule method
Objectives	Verify that correct vaccine schedule is generated based on child's DOB.
Pre-conditions	Vaccine data is stored in the database.
Test steps	Enter Date of birth. Turn on the auto-schedule toggle button. Call generates auto-schedule. Check returned schedule list after hospitals approval.
Input	DOB="2024-01-01"
Expected result	Schedule is created based on the child's age
Actual Result	Pass
Status	Pass

5.1.5 Test case5:

Table 5-5: Unit test case 5

TC-ID	UT-5
Req.ID	R17
Title	Reminder function, email dispatched.
Description	Test the reminder method for notification.
Objectives	Ensure that the reminder method send notification to the correct address.
Pre-conditions	Parent email exist in the system. SMS/Email services configured in the system.
Test steps	Call sendReminders (email, phone no, appointment date). Check email services. Verify email received.
Input	Email= hllohifza2026@gmail.com password="123456789" appointment date="ABC".
Expected result	System dispatch e-mail and appointment details, log shows 200 Ok from SMTP.
Actual Result	Pass
Status	Pass

5.1.6 Test case6:

Table 5-6: Unit test case 6

TC-ID	UT-6

Req.ID	R13
Title	Vaccine stock, management(add/remove).
Description	Test the add () and remove () method in the vaccine stock class
Objectives	Verify that vaccine can successfully added and remove from stock by hospitals.
Pre-conditions	Hospital is logged in and approved by the admin.
Test steps	Call adds (name, quantity) to add a vaccine. Call delete (name, quantity) to delete the vaccine.
Input	Vaccine name= "polio" Quantity=10. Add
Expected result	Vaccine added and removed from hospitals stock.
Actual Result	Pass
Status	Pass

5.1.7 Test case7:

Table 5-7: Unit test case 7

TC-ID	UT-7
Req.ID	R10
Title	Rating calculation
Description	Test the calculateRating () method
Objectives	Ensure average rating is calculated from multiple user ratings.
Pre-conditions	Atleat 3 Rating is submitted by parent in case of satisfaction.
Test steps	Submit rating. Call calculateRating () Notify returned average rating
Input	Rating 3 stars.
Expected result	Returned average rating.
Actual Result	Pass
Status	Pass

5.1.8 Test case8:

Table 5-8: Unit test case 8

TC-ID	UT-8
Req.ID	R6
Title	Forgot password, OTP e-mail.
Description	Test forgot password () method

Objectives	Ensure a code is generated and send to the user's e-mail and user can reset its password.
Pre-conditions	Users e-mail exists in the system.
Test steps	Check if code is sent via e-mail. Verify code validity/expiry.
Input	Email= hllohifza2026@gmail.com
Expected result	User receive a reset code and successfully reset password.
Actual Result	Pass
Status	Pass

5.2 Functional Testing

Functional testing test that the functions of software or system. It happens to check that each function performs the designated task correctly or not.

5.2.1 Test case 1:

Table 5-9: Functional test case 1

TC-ID	FR1
Req.ID	R2
Title	View vaccine info without login.
Description	A guest user can browse vaccine information without signing up.
Objectives	Verify that vaccine information is publicly accessible.
Pre-conditions	Website is running; vaccine; data is loaded in the database.
Test steps	Open the website homepage. Click on vaccine info. Observe displayed information.
Input	No login required; user is as a guest.
Expected result	Vaccine names, description and dosage schedules displayed without authentication.
Actual Result	Vaccine info page loaded full details for all age groups.
Status	Pass

5.2.2 Test case 2:

Table 5-10: Functional test case 2

TC-ID	FT2
Req.ID	R1
Title	View vaccination guidelines

Description	Users can read awareness content and guidelines without logging in.
Objectives	Ensure the guidance module renders correctly for unauthenticated users.
Pre-conditions	Guidance content is present in the database.
Test steps	Negative to the guidance. Read the content
Input	Guest user; no authentication.
Expected result	Guidance page available with full content about vaccine importance.
Actual Result	Guidance section displayed completely in both English and Urdu.
Status	Pass

5.2.3 Test case3:

Table 5-11: Functional test case 3

TC-ID	FT3
Req.ID	R8
Title	Book appointment for child
Description	A logged in parent fills and submit the child registration form to book an appointment.
Objectives	Verify appointment booking workflow for a parent.
Preconditions	Parent is logged in; at least one hospital with available slots exists.
Test steps	Navigate to the Child Form Enter child details (name, DOB, gender, vaccine service type, request type) Select hospital from dropdown. Upload birth certificate and picture. Click submit.
Input	Child: name=”ABC” DOB= 2024-03-10” Gender= “Female” Vaccine= “BCG” Hospital= “ABC”.
Expected result	Appointment submitted; success message shown record stored in DB.
Actual Result	Appointment has been taken successfully.
Status	Pass

5.2.4 Test case4:

Table 5-12: Functional test case 4

TC-ID	FT4
Req.ID	R10
Title	Select Hospital from available slots
Description	Parent can view available hospital slots and select one.
Objectives	Verify that available slots are fetched, displayed correctly.
Pre-conditions	Hospital has at least 2 available slots configured.
Test steps	Click hospital dropdown in child form. Observe available slots. Select a hospital slots. View hospital profile.
Input	Hospital: ABC hospital Slots 3
Expected result	Slots displayed: parent can select one hospital profile visible.
Actual Result	Pass
Status	Pass

5.2.5 Test case5:

Table5-13: Functional test case 5

TC-ID	FT5
Req.ID	R12
Title	View the vaccine schedule
Description	Parent can view upcoming vaccine schedule for their child.
Objectives	Confirm vaccine overview module shows correct schedule and past records.
Pre-conditions	Child has at least one appointment approved; hospital has updated the schedule.
Test steps	Login as parent. Go to vaccine schedule& record. Tap View schedule and record.
Input	Parent account with one child; one approved appointment.

Expected result	Upcoming vaccine name, hospital shown, date, past completed vaccines listed.
Actual Result	Schedule and records update with correct dates and names.
Status	Pass

5.2.6 Test case6:

Table 5-14: Functional test case 6

TC-ID	FT6
Req.ID	R16
Title	Hospital Registration and Admin approval.
Description	A hospital registers and waits for the admin approval before accessing the dashboard.
Objectives	Ensure hospital registration workflow and admin approval gate work correctly.
Pre-conditions	Admin is logged in; Hospital has not yet registered.
Test steps	Hospital registers or signup form. Admins log in and open Manage Hospitals. Admin approves the hospital Hospital logs in and verifies dashboard access.
Input	Hospital:" ABC" Email= email@gmail.com Type=" private" Charges=500 Certificate=(image).
Expected result	Hospital registered; Admin sees request; Approval grants and dashboard access.
Actual Result	Pass
Status	Pass

5.2.7 Test case 7:

Table 5-15: Functional test case 7

TC-ID	FT7
Req.ID	R14
Title	Hospital manages appointment slots
Description	Hospital can set and update the number of available appointment slots.

Objectives	Verify slots management functionality for hospitals.
Pre-conditions	Hospital is approved and logged in.
Test steps	Navigate to manage slots. Enter number of slots and time. Click save. Update slots to different value.
Input	Select Type (Home, Hospital), Time, Days, No. of slots
Expected result	Slots saved to database
Actual Result	Slot data saved and updated correctly
Status	Pass

5.2.8 Test case 8:

Table 5-16: Functional test case 8

TC-ID	FT8
Req.ID	R17
Title	Admin manages hospitals and Rating
Description	Admin can view the hospitals rating and eliminate poorly rated hospitals.
Objectives	Confirm admin panels hospital rating management and action features.
Pre-conditions	Amin is logged in hospitals have received user ratings.
Test steps	Admin navigates to view rating. Clicks eliminate hospital on low rated hospital. Notification sent to the hospitals.
Input	Hospital with rating 1 out of 5 stars.
Expected result	Admin sees rating and eliminates the hospitals
Actual Result	Pass
Status	Pass

2.3 Integration Testing

Integration testing perform testing by combining different modules and check how they interact with each other.

5.3.1 Test Case 1

Table 5-17: Integration test case 1

TC-ID	TC1
Req.ID	R3
Title	Frontend->Backend-> database Parent registration
Description	Test the integration path from submission of registration parent fill it.
Objectives	Verify that all the data stored in database correctly
Pre-conditions	Fast Internet is required. Database must active.
Test steps	The account is not created with using e-mail. Click on parent registration enter details to fill form and submit it.
Input	Username: Abcde e-mail: abs@gmail.com contact: +92abc
Expected result	Parent has been registering successfully, data stored in database
Actual Result	Pass
Status	Pass

5.3.2 Test Case 2

Table 5-18: Integration test case 2

TC-ID	IT2
Req.ID	R15
Title	Appointment Booking→hospital approval
Description	Test the integration path from parent appointment request to the hospital approval.
Objectives	Verify that appointment booking and hospital approval system work correctly.
Pre-conditions	Parent must be logged in to book an appointment.
Test steps	Parent click on “Add child” to take appointment. Request sent to the hospital. Hospital clicks on approve to approve the appointment that save in database notify to users.
Input	Child name: ABC Hospital: ABC
Expected result	Child is Successfully Registered.
Actual Result	Pass

Status	Pass

5.3.3 Test Case 3

Table 5-19: Integration test case 3

TC-ID	IT3
Req.ID	R9, R12
Title	Auto schedule→Vaccine overview
Description	Test that generated auto-schedule.
Objectives	Verify that the auto-schedule is generated accurately.
Pre- conditions	Child data must be added in system to enable auto schedule.
Test steps	Parents enter the child's DOB. Turn on the auto schedule for enabling it. Auto-schedule is generated and parents view it after hospital approval.
Input	DOB= ''2025-01-01''
Expected result	Auto schedule is generated successfully and parent can view it.
Actual Result	Pass
Status	pass

5.3.4Test Case 4

Table 5-20: Integration test case 4

TC-ID	IT4
Req.ID	R11
Title	Hospital schedule-> update parent record.
Description	Test that when the hospital marks vaccine as completed, it updates in parents' vaccine records.
Objectives	Verify that vaccine record shows to the parent accurately.
Pre- conditions	Appointment must be approved. Parent must be login. Hospital account is active.
Test steps	Hospital opens the system. Hospital marks the vaccine as completed. Also enter the next vaccine date. Parent view his record.

Input	Vaccine Status: Completed Next Date:01-06-2026
Expected result	Updated information will show in parent record. Parents will view the next date of vaccine.
Actual Result	Pass
Status	Pass

5.3.4 Test Case 5

Table 5-21: Integration test case 5

TC-ID	IT-5
Req.ID	R4, R16
Title	Hospital Registration->Admin panel
Description	Test the integration of hospital registration to their approval by admin.
Objectives	Verify that hospital registration and admin approval work correctly.
Pre- conditions	Admin account must exist. Hospital not registered yet.
Test steps	Hospital fills the registration form. Admin approve the hospital request. Hospital login the system.
Input	Hospital name: ABC Type: Private
Expected result	Hospital is approved. Hospital accesses the dashboard.
Actual Result	Pass
Status	Pass

5.3.5 Test Case 6

Table 5-22: Integration test case 6

TC-ID	IT-6
Req.ID	R10, R17
Title	User rating-> Admin Rating management.
Description	Test the workflow of rating submitted by parent to Hospitals and managed by admin.

Objectives	Verify that rating system work properly.
Pre- conditions	Parent must be logged in.
Test steps	Parent submit rating (1-5). Admin check rating. Admin take action based on low rating.
Input	Rating: 3 Stars.
Expected result	Rating saves in database Rating show to admins
Actual Result	Pass
Status	Pass

5.4 Performance Testing

Performance testing tests the efficiency and stability of software. It also happens to check that how much fast the system completes its task.

5.4.1 Test Case 1:

Table 5-23: Performance test case 1

TC-ID	PT-1
Req.ID	R5
Title	Login response time
Description	Response time of the log in APIs under normal load.
Objectives	Ensure login APIs respond within 3-4 seconds.
Pre- conditions	Frontend and backend servers running locally, browser DevTools configured
Test steps	Login through the application in the browser. Open Chrome DevTools (Network tab). Record the response time of the login request. Repeat the test 5 times and calculate average.
Input	Valid credentials.
Expected result	Average response time is less than 0.4 seconds, no error occurs
Actual Result	Average response time = 0.163 seconds (163 ms), no errors occurred
Status	Pass

5.4.2 Test Case 2:

Table 5-24: Performance test case 2

TC-ID	PT-2
Req.ID	R4
Title	Hospital signup response time.
Description	Response time of hospital signup API including certificate file upload.
Objectives	Ensure hospital signup completes within a reasonable time despite file upload.
Pre- conditions	Server running, Cloudinary configured, browser DevTools open.
Test steps	Submit the Hospital Signup form with a certificate file attached. Record response time of the hospital-signup request via browser DevTools (Network tab). Repeat 3 times and calculate average.
Input	Valid hospital details with certificate file
Expected result	Response time < 10 seconds
Actual Result	Average response time = 8.27 seconds (7.51, 8.83, 8.47)
Status	Pass

5.4.3 Test Case 3:

Table 5-25:Performance Test case3

TC-ID	PT-3
Req.ID	R9
Title	Auto schedule generation speed. (measured via child registration)
Description	To measure the time taken to generate the vaccine schedule automatically, which happens as part of the child registration process when auto-schedule is enabled.
Objectives	Ensure the schedule generation (within register-child) completes within a reasonable time
Pre- conditions	Server running, frontend and backend connected, hospital and vaccine data already existing in database.
Test steps	Register a child through the Add Child form with auto-schedule enabled (this internally triggers schedule generation). Record response time of the register-child request using browser DevTools (Network tab). Repeat 5 times and calculate average.
Input	Valid child details with date of birth and auto-schedule = true
Expected result	Response time < 6 seconds

Actual Result	Average response time = 5.03 seconds (5.35, 4.68, 4.59, 5.07, 5.45)
Status	Pass

5.4.4 Test Case 4:

Table 5-26: Performance Test case 4

TC-ID	PT-4
Req.ID	R10
Title	Sending email notifications.
Description	Measure the time taken for the admin notification email to be delivered upon a new hospital signup.
Objectives	Ensure notifications are delivered promptly without failures
Pre- conditions	Gmail SMTP configured correctly, admin email account accessible
Test steps	Submit the Hospital Signup form. Immediately check the admin's email inbox. Note the time difference between registration and email arrival. Repeat 3 times.
Input	3 hospital signups with valid details
Expected result	All notification emails delivered within 10 seconds, 0 errors occur
Actual Result	Emails were received almost instantly (within 1-2 seconds) for all 3 signups, no failures occurred
Status	Pass

1.5 Summary

This chapter covers the integration, functional, unit and performance testing. It ensures that overall system is stable, fast and efficient. It also describes that how the features work and integrate with each other

Chapter 6

System Conversion

6 Introduction:

System conversion is the process of transferring the existing traditional system into the digital Kids Vaccination system. It involves changing in vaccination record, taking booking, reminders, schedule and other features to operate the new system. Developing system ensures the security, efficient use of system and healthcare services for users.

6.1 Conversion Method:

For developing Kids Vaccination Booking the selected conversion method is implemented:

6.1.1 Pilot Conversion:

Pilot conversion method is used because system is implemented in limited areas before it is deployed. In our project it is used because Kids Vaccination Booking is initially tested in limited environment like in a single clinic or hospital ensuring the fully testing before implementation. After that the project is implemented in multiple areas.

Justification:

- **Reduced risk:** Initially, small group uses the system which may reduce the risk in wide failure:
- **Focus testing:** Every module like parent registration, hospital registration, child management and reminders are testing successfully before implementation.
- **User feedback loop:** User ratings and appropriate action taken by the admin improve the system's performance.

Pilot scope:

- **Initial group:** Selected clinics and parents.
- **Modules tested:** Addchild, manage schedule and reminders.
- **Duration:** Two weeks are required for testing.

6.2 Deployment:

Deployment is the process of launching and transferring the kid's vaccination booking system from developing environment to operational environment by making it accessible for users. This includes developing the secure website, ensuring database services and all the functions work correctly for Android mobile and other android devices.

Table 6-1:Deployment

Step no	Task	Details
1.	Build website	React.js is used to generate the website.
2.	Backend configures	Node.js and Express.js are used to configure backend services.
3.	Database setup	MongoDB is used as database to save all records.
4.	Authentication	JWT is used for authentication like login.

5.	Web deployment	Deploy the frontend and backend of website on hosting platform.
6.	Device testing	Website run on different devices and browsers for testing.
7.	Live verification	Uses APIs for reminders
8.	Deployment	System fully deploys for users after testing.

1.2.1 Data Conversion:

Table 6-2: Data Conversion

Step no.	Task	Details
1.	Data extraction	Collect child information from parent and hospitals information from hospitals and collect the vaccine records etc.
2.	Data cleaning	Remove the inconsistency and many other problems by updating duplicating, missed records and invalid records.
3.	Data transformation	Convert the data in MongoDB compatible format.
4.	Data Loading	Upload and store the updated data in database.

6.2.2 Training:

Give the instruction to the users step-by-step in user guidance about how the website will operate and used. Here the instructions about two modules are given:

Table 6-3: Instruction 1

Step	Instructions (AddChild)
1.	Login by using e-mail and password.
2.	Tap on the “Addchild” from dashboard for booking.
3.	Add child details like name, DOB, service type booking type address etc.
4.	At the end tap on “submit”.

Table 6-4: Instruction 2

Step	Instruction (View schedule)
1.	Tap on “view schedule”
2.	View the upcoming vaccine based on DOB
3.	View the completed vaccine.

4.	Receive reminders for the upcoming vaccine.

6.3 Post Deployment Testing:

Post deployment testing is performed to check that all the functions and modules working properly after deployment. The objective is to ensure the features functionality, system reliability and security after deployment.

Table 6-5: Post Deployment testing

Step	Test Activity	Expected Outcomes
1.	Launch website	The web application kids vaccination booking is launched properly without any errors.
2.	Login functionality	Users are able to login by using e-mail and passwords.
3.	Role-based access	Test that Admin, Hospital and parent can access the functionalities based on their roles.
4.	Child management	Parents can add their child by entering their details and also manage their profiles successfully.
5.	Performance	Test that the system responds successfully with in minimum time.
6.	Vaccine overview	Parents can view the vaccine schedules with upcoming and received vaccine. Parents can also see the records successfully.
7	Schedule management	Hospitals can manage the schedule of child's vaccine successfully.
8.	Manage slot/stock	Hospital can update and manage the vaccine stock and their slots.
9.	Data integrity	Child profiles, Hospital features work correctly after login session.
10.	Reminders	Hospitals and parent receive reminders successfully.

6.4 Challenges and solutions:

During deployment and post deployment of Kids vaccination booking several technical and operational challenges are faced. Here several challenges along with their solutions are listed:

Table 6-6: Challenges and solutions

Challenges	Description	Solution
User navigate difficulties	Some parents do not understand the features of website and face difficulties in operating it.	A detail instruction about website is provided in provide user-guidance.
Reminder issues	Some parents do not receive reminders on time	Reminder system is configured properly.
Responsive design	Websites pages are not displayed correctly on small screens	Wen design techniques are implemented to improve website performance.
Login issues	Some users do not remember their password for login	To resolve this forgot password option is used.
Vaccine record	Some users enter invalid or wrong users' details.	In this case form validation is implemented to ensure all the fields are filled to work correctly.

6.5 Summary:

This chapter focuses on the deployment of project. This ensures the features functionality, security and ensure that all the features work correctly because all the features are tested before implementation.

Chapter 7

Conclusions

Introduction

This chapter discuss the conclusion of Kids vaccination booking. It evaluates which project objectives and goals have successfully achieved. Moreover, this chapter highlight the limitations of current system and suggest possible improvement along with future enhancement that can be applied to further enhance system performance, functionality and overall system.

7.1 Evaluation

This section describes that all the objectives have been completed that are describes in FYDP.

Table 7-1 Objectives with status

OBJECTIVES	Status
Design a platform where users can **book and take appointments** for their child vaccination in desired hospitals.	Completed
Allow hospitals to post detailed information about **vaccination stock** that might helpful for parents in booking.	Completed
Provide more hospital options **(real time slots)** to parents for improving their child vaccination.	Completed
Establish a simple and well-organized **booking system.**	Completed
Provide the facility of **hospital services** Govt. services for the people who can't afford private dues.	Completed
Provide the facility of saving previous records.	Completed
Gain trust in parents through **verification** for team.	Completed
Hospital also **manages slots.**	Completed
User gives **Rating** to **hospitals** and **views the hospital profile.**	Completed
Send **reminders**	Completed
Admin can also **manage the appointments** of hospitals users	Completed
Admin **view Rating** and take action against hospitals based on low rating	Completed
Parents can **view the schedule** that is Upcoming vaccines.	Completed
Hospital **manages the schedule** by marking when vaccine completed.	Completed

7.2 Traceability Matrix

A traceability matrix is a table used in testing to track each requirement is tested properly. It also analyzes that how different requirements link with test cases.

Table 7-2: Traceability matrix

Requirement ID	Requirement Description	Design specification	Code	Test ID
R1	User can view overall vaccine info based on child's age	Vaccine info	vaccineController.js	FT1
R2	User can create account by entering details.	ParentSignup	parentController.js	UT1
R3	Hospital can create account	HospitalSignup	hospitalController.js	FT6
R4	User can login the system	Login	authController.js	UT2
R5	User can reset their password	Forgot password	forgetpasswordController.js	UT8
R6	Parent can add their child	AddChild	parentController.js Child.js	IT2
R7	Automatically schedule is generated	Auto schedule	Child.js parentController.js	UT4
R8	User can rate the hospital	Rating	parentController.js	UT7
R9	User can view the schedule of child.	ViewSchedule	parentController.js hospitalController.js	FT5
R10	Hospitals can manage slots availability	ManageSlots	hospitalController.js	FT7
R11	Hospitals can manage the parents' request.	View Appointment	hospitalController.js parentController.js	IT2
R12	Amin can manage the hospitals request	Manage hospital	adminController.js	FT8
R13	Admin can take action against rating.	View Rating	adminController.js	IT6
R14	Hospital can manage their stock	Manage Stock	hospitalController.js	FT7

7.3 Conclusion

Kids Vaccination booking fulfills all the objectives defined at start of the project. It provides an efficient platform for managing child vaccination process include secure authentication, booking in desired hospitals, timely reminders, vaccine schedule and rating. The project contributes to improving health care service efficiency and ensure better vaccination management.

Table 7-3: Objectives with functionalities

ID	Objective	Functionality Provided
1	User Registration	Authentication Module (signup, login, forgot password)
2	Child Registration	AddChild
3	Vaccine Stock Management	Manage Stock Module (update stock)
4	Multiple Hospital Options	Manage Slots (slot check)
5	User friendly platform	Vaccine Guidance, Vaccine Info
6	Vaccine Schedule generated automatically	Auto Schedule Module
7	Rate Hospital	Rating Module
8	Hospital performance evaluated by Admin	Manage Rating
9	Hospital access requires admin approval	Manage Hospital (Approve/rejects)

7.4 Future Work

After completion of pour project, we will focus on developing mobile app version of our system.

Payment gateway for making online payment and many more features will be added.

References

Book:

Sommerville, I. (2015). Software Engineering. In *Pearson Education* (pp. 120-321).

Web Resources:

Husnain, M. (n.d.). *SOW Programming*. Retrieved from Youtube: https://www.youtube.com/@SOWProgramming

Khan, H. A. (n.d.). *Code with Harry*. Retrieved from Youtube: https://youtube.com/@codewithharry

Appendix A

Use case Description

Appendix-A Use case Description

Use case 1

ID	UC-1
Name	Guidance
Actors	End users
Description	This use case gives a brief description about the importance of vaccines
Trigger	User select the Guidance on home page
Preconditions	User must have access/open the website
Post condition	A complete guidance is shown on the screen User can read it.
Normal flow	4. User open the website. 5. Select the “Guidance” option on homepage. 6. Guidance screen appears.
Alternative flow	If the user has not stable network. Then the website will not show the page and msg show please check your network connection.
Business flow	Data us fetched from the database and shown on screen.
Assumptions	User must have stable internet connection. Database must be active for fetching and displaying data.

Use case 2

ID	UC-2
Name	Vaccine info
Actors	End users
Description	This use case gives a brief description about the doses of vaccines
Trigger	User select the Vaccine info on home page
Preconditions	User must have access/open the website
Post condition	A complete vaccine schedule is shown on the screen User can read it.
Normal flow	4. User open the website. 5. Select the “Vaccine info” option on homepage. 6. Guidance screen appears.

Alternative flow	If the user has not stable network. Then the website will not show the page and msg show please check your network connection.
Business flow	System fetched from the database and shown on screen.
Assumptions	User must have stable internet connection. Database must be active for fetching and displaying data.

Use case 3

ID	UC-3
Name	Parent Signup
Actors	End users
Description	This use case allows users to enter their email, strong password and their number
Trigger	The user opens the website and c lick on parent sign up on the screen.
Preconditions	User should not be registered already
Post condition	A new user account is created in database.
Normal flow	7. User open the website. 8. A dashboard is appeared. 9. User click the parent sign up. 10. User enter their credentials like e-mail, name, phone number and a strong password 11. Anew account is created in database. 12. User is redirected to the login screen.
Alternative flow	If the e-mail is registered, an error message will appear. If the user does not enter a strong password message shown to correct it.
Business flow	E-mail must be unique. Password must be unique like a mix of numbers/ characters, special numbers.
Assumptions	User must have internet access.

Use case 4

ID	UC-4
Name	Hospital Signup
Actors	End users

Description	This use case allows users to enter their email, strong password and their number
Trigger	The user opens the website and click on Hospital sign up on the screen.
Preconditions	User should not be registered already.
Post condition	A new hospital account is created in database.
Normal flow	7. User open the website. 8. A dashboard is appeared. 9. User click the hospital sign up. 10. User enter their credentials like e-mail, name, phone number, a strong password, certificate, slots and address. 11. A new account is created in database. 12. User is redirected to the login screen.
Alternative flow	If the e-mail is registered, an error message will appear. If the user does not enter a strong password message shown to correct it.
Business flow	E-mail must be unique. Password must be unique like a mix of numbers/ characters, special numbers.
Assumptions	Hospital must have internet access.

Use case 5

ID	UC-5
Name	Login
Actors	User
Description	This use case allows users to login e-mail and password.
Trigger	Users click on the login on the dashboard.
Preconditions	User must be already registered.
Post condition	If the e-mail and password is correct user redirected to their panel. If e-mail and password is invalid an error appears.
Normal flow	5. User open the website a dashboard appears. 6. Click on the login features. 7. Enter email and password. 8. If valid, user can access to their panel
Alternative flow	If the email is invalid an error message is shown.

Business flow	Email and password must be valid. User must be registered already.
Assumptions	User must have stable internet connectivity. User must have knowledge.

Use case 6

ID	UC-6
Name	Forgot password
Actors	End users
Description	This use case allows users to reset their password via email
Trigger	User click on the forgot password.
Preconditions	User must be registered.
Post condition	User reset passwords and can get access to their panel.
Normal flow	5. User click on “forgot password”. 6. User enter register email. 7. A code is sent to the email. 8. User enter this code and then user redirected to their dashboard.
Alternative flow	If the user enters the unregistered email, then an error message will show.
Business flow	Email must be valid.
Assumptions	User must have stable internet connection.

Use case 7

ID	UC-7
Name	Setting
Actors	End users
Description	This use case allow user to change their name and passwords.
Trigger	User click on the setting on their dashboard.
Preconditions	User must already be registered
Post condition	Users’ data is undated.

Normal flow	6. Users tap on “setting”. 7. User tap on “edit” 8. A screen appears. 9. User update his information. 10. Updated information is stored in database.
Alternative flow	If the internet connection is not stable then data is not updated and message “check your connection” is shown.
Business flow	The updated data must be validated.
Assumptions	User must have stable internet connection. User must be already registered.

Use case 8

ID	UC-8
Name	Add child
Actors	End user
Description	This use case allows users to register their child by filling form and entering required child’s credentials
Trigger	User select the “Add Child” on the dashboard.
Preconditions	Parent must be login into the system.
Post condition	Child is register and schedule is created automatically.
Normal flow	5. User tap on “Add Child” 6. A child form appears 7. User enters child’s name, DOB, select service type, select the hospital from dropdown, click on auto schedule, upload child’s picture and birth certificate. 8. Click on submit.
Alternative flow	If user enter incomplete information, then the error message shown “Enter complete information” If the child is already registered then error message shown.
Business flow	Childs data must be accurate.
Assumptions	User has stable internet connection for submitting child’s form

Use case 9

ID	UC-9
Name	Vaccine Overview
Actors	End user
Description	This use case allows users to view schedule and record of their child vaccination course.

Trigger	User select the “Vaccine Overview” on dashboard.
Preconditions	Child must be registered.
Post condition	The complete child’s vaccine record is displayed with upcoming and completed vaccine.
Normal flow	4. Users tap on the “Vaccine Overview” 5. A screen is displayed with complete vaccination course. 6. User can view the upcoming vaccine with date.
Alternative flow	If the child is not registered then the schedule is not displayed. If the hospital does not update the schedule, then the schedule is snot updated.
Business flow	Child vaccine data is updated by the hospital. This stored in database Database update data in parents’ schedule And displayed to the user.
Assumptions	Data must be updated by hospital. Database must be active to show the updated data

Use case 10

ID	UC-10
Name	Manage stock
Actors	End user (Hospital)
Description	This use case allows hospitals to enter their vaccine stock. Hospital can also update it.
Trigger	Hospitals click on the “Manage Stock”” on their dashboard.
Preconditions	Hospitals must be registered and login.
Post condition	Vaccine stock screen displayed where the hospitals enter their stock.
Normal flow	5. Hospital taps on “Manage stock” 6. Vaccine stock screen appears 7. Hospitals enter vaccine stock details 8. This data is stored in database.
Alternative flow	If the internet is not stable then data is not stored in database.
Business flow	System store data in database after entering the data.
Assumptions	Hospitals must have stable internet connection Database must be activated to store data.

Use case 11

ID	UC-11
Name	View Rating
Actors	Hospital
Description	This use case allows hospitals to view the rating submitted by users.
Trigger	Hospital clicks on View Rating on the dashboard.
Preconditions	Hospital must be login and rating is submitted by parents.
Post condition	A screen appeared where all the stock of vaccine is shown.
Normal flow	4. Hospital taps on the vaccine stock. 5. A screen of rating appears. 6. Hospital can view it
Alternative flow	If user enter invalid data (negative number) the system shows error message.
Business flow	Accurate data must be entered.
Assumptions	User must have already stock.

Use case 12

ID	UC-12
Name	Manage Slots
Actors	Hospital
Description	This use case allows hospitals to manage their slots.
Trigger	Hospital clicks on manage slots on the dashboard.
Preconditions	Hospital must be registered and logged in.
Post condition	A screen appeared where hospital can add their slots.
Normal flow	5. Hospital taps on the manage slots. 6. Hospital enters No. of slots and days availability. 7. These new slots are stored in database 8. Hospital can also update it.
Alternative flow	If user enter invalid data (negative number) the system shows error message.
Business flow	Accurate data must be entered.
Assumptions	Hospitals must have stable internet connection Database must be activated to store data.

Use case 13

ID	UC-13

Name	Manage Appointment
Actors	Hospitals
Description	This use case allows users to manage the parents' appointment request.
Trigger	Hospital clicks on "manage appointment".
Preconditions	Hospital must be login the system.
Post condition	A screen with a lot of appointments displayed.
Normal flow	6. Hospital admin clicks on manage appointments. 7. A list of appointment / request is displayed on the screen 8. Hospital admin can approve or reject the request. 9. The system updates the status in the database. 10. Database send notification to user regarding decision.
Alternative flow	If the internet is not stable, status is not updated in database. And database does not send notification.
Business flow	System must update data in database.
Assumptions	Appointment/ request must be submitted by the users Hospital must have stable internet connection.

Use case 14

ID	UC-14
Name	Manage schedule
Actors	Hospitals
Description	This use case allows hospital to manage the child's vaccine schedule.
Trigger	Hospital clicks on the vaccines schedule on dashboard.
Preconditions	Hospitals must be logged in. Childs vaccine stock must be existed already in the system
Post condition	Hospitals updated the schedule and data is stored in the database.
Normal flow	6. Hospital admin taps on manage schedule 7. Schedule of multiple children is displayed on screen. 8. If vaccine is given staff marked it read as completed 9. Hospital staff enter the next due date for upcoming vaccine. 10. System saves the updated data successfully.
Alternative flow	If the staff enter invalid data (past date) then system shows error message If the network is not stable then updated data is not stored in database.
Business Flow	System stored the updated data in database. System automatically updates the child's vaccine schedule after changes made by hospital.

Assumptions	Hospital must have permissions to change the schedule.

Use case 15

ID	UC-15
Name	Manage hospital
Actors	Admin
Description	This use case allows users to approve or reject hospitals request
Trigger	Admin tap on the manage hospitals on their dashboard.
Preconditions	Admin must be logged in.
Post condition	Admin verified the hospital and granted request to the system.
Normal flow	5. Admin tap on manage hospitals request. 6. A screen with the hospital request will display. 7. Admin made a decision through verification. 8. System stores the decision approve or reject in the database.
Alternative flow	If the internet connection is not stable, decision is not stored in the database.
Business flow	Hospital data must be validated before granting access.
Assumptions	Internet connection must be available during verification.

Use case 16

ID	UC-16
Name	Manage Rating
Actors	Admin
Description	This use case allows admin to evaluate hospital performance and made decision based on the performance.
Trigger	The admin clicks on the view rating on their dashboard.
Preconditions	Admin must be logged in in the system.
Post condition	Admin evaluating the hospital performance through users rating.
Normal flow	6. Admin taps on manage hospitals performance. 7. A screen with hospital along with their rating displayed. 8. Admin evaluate the performance. 9. Deactivate the hospital based on their performance if the rating is below the 3 stars. 10. System stores the data in the database.

Alternative flow	If no users rating is available admin will not be able to evaluate the performance.
Business flow	System stores the updated data. System sends the notification the hospitals.
Assumptions	User rating must be existed in the system.

Appendix B

Coding Standard

Appendix B: General Coding Standards & Guidelines

To ensure maintainability, readability, and scalability of the **Vaccine Management System**, the following coding standards were followed:

1. Variable Naming Conventions

Conventions:

- camelCase → variables & functions (e.g., sendNotification, generateSchedule, hospitalImg, vaccinationImg)
- PascalCase → React component names (e.g., ParentDashboard, HospitalLayout, AdminLayout, VaccineSchedule)
- UPPER_SNAKE_CASE → environment variables & constants (e.g., VITE_API_URL, MONGO_URI)

2. Visual Scope Indicators

Clear scope indicators were used across the codebase:

- showSplash, setShowSplash → local component state (React useState)
- .env variables (e.g., VITE_API_URL) → clearly separated from application logic
- Environment-specific configuration kept outside version-controlled source files

3. Read-Only and Immutable Fields

- Constant configuration values (e.g., API URLs, MongoDB connection strings) are defined once via environment variables and never mutated at runtime.
- Sensitive credentials (SMS/Email API keys, database URIs) are never hardcoded; they are injected through `.env` files and Railway/Vercel environment variable settings.

4. Function Naming and Behavior

Functions clearly describe their purpose:

- sendSMS()
- sendEmail()
- sendNotification()
- generateSchedule()
- vaccineReminderCron()

5. Commenting Style

Comments are added only where necessary to clarify non-obvious logic, e.g.:

js

```
// Injects environment variables and starts the notification worker
```

6. Indentation and Code Structure

- Consistent indentation (2–4 spaces) maintained throughout.
- Proper use of `{}` in all control structures.
- Deep nesting avoided by breaking logic into smaller utility functions (sendSMS.js, sendEmail.js, generateSchedule.js).

7. File and Folder Naming

File names match their responsibility/module, and folder structure separates concerns clearly:

Backend:

backend/

- controllers
  - authController.js
  - parentcontroller.js
  - hospitalcontroller.js
  - vaccinecontroller.js
  - notificationcontoller.js
  - admincontroller.js
- routes
  - authroutes.js
  - parentroutes.js
  - hospitalroutes.js
  - vaccineroutes.js
  - notificationroutes.js
  - adminroutes.js
- middleware
  - authmiddleware.js
  - uploadmiddleware.js
- models
  - Admin
  - Child
  - Hospital
  - Notification
  - Parent
  - Vaccine
- Seed
  - Seedadmin.js
  - Seedvaccine.js
- utils

- sendSMS.js
- sendEmail.js
- sendNotification.js
- generateSchedule.js
- vaccineReminderCron.js

server.js

Frontend:

frontend/src/

- Components/
  - AdminLayout/
  - HospitalLayout/
  - ParentLayout/
  - FloatingMenu/
  - Footer/
  - Notificationbell/
  - PageHero/
  - Splashscreen/
- Pages/
  - HomePanel/
  - ParentPanel/
  - HospitalPanel/
  - AdminPanel/
  - About/
- styles/
- assets/

8. Class Structure & Separation of Concerns

Each layer has a single, well-defined responsibility:

- **Controllers** (authController.js) → handle incoming requests and responses
- **Routes** (authRoutes.js) → define API endpoints and map them to controllers
- **Utils** (sendSMS.js, sendEmail.js, sendNotification.js) → reusable business logic and third-party integrations
- **Components/Pages** (frontend) → UI is separated into reusable Components (layouts, shared widgets) and route-specific Pages (Parent, Hospital, Admin, Home panels)

9. Access Modifiers and Scoping

- Environment-sensitive values (API keys, database URIs) are scoped strictly to the backend/server-side and never exposed to the frontend bundle.
- Only necessary configuration (e.g., VITE_API_URL) is exposed to the client via Vite's import.meta.env.

10. Cohesion and Coupling

High cohesion:

- Notification-related logic is grouped together (sendSMS.js, sendEmail.js, sendNotification.js)
- Panel-specific pages are grouped under their respective folders (ParentPanel, HospitalPanel, AdminPanel)

Low coupling achieved by:

- Modular folder structure (Components vs. Pages vs. Utils)
- Frontend communicates with backend only through a single configurable API base URL (VITE_API_URL), decoupling frontend code from any specific backend deployment

11. DRY Principle

Reusable modules avoid code duplication:

- sendNotification.js centralizes notification dispatch logic used by multiple controllers
- Shared layout components (AdminLayout, HospitalLayout, ParentLayout) avoid repeating UI structure across panel pages
- Shared stylesheets (Adminshared.css, Hospitalshared.css, ParentShared.css) avoid duplicating CSS across panel-specific pages

12. Single Responsibility Principle (SRP)

- Each utility function performs one task (e.g., sendSMS() only sends SMS, generateSchedule() only computes vaccine schedules)
- Each route file handles one domain of the application (authRoutes.js → authentication only)

13. Exception Handling and Logging

- Errors are handled using try-catch blocks in async operations (database calls, SMS/email dispatch).
- Deployment logs are monitored via Railway (backend) and Vercel (frontend) build/runtime logs to catch module resolution errors, missing environment variables, and runtime crashes early.

Appendix C

Prototype

Appendix C: Application Prototype

1. Prototype Overview

The Application prototype provides a visual representation of the Kids Vaccination Booking system's layout, behavior. It covers the following core modules:

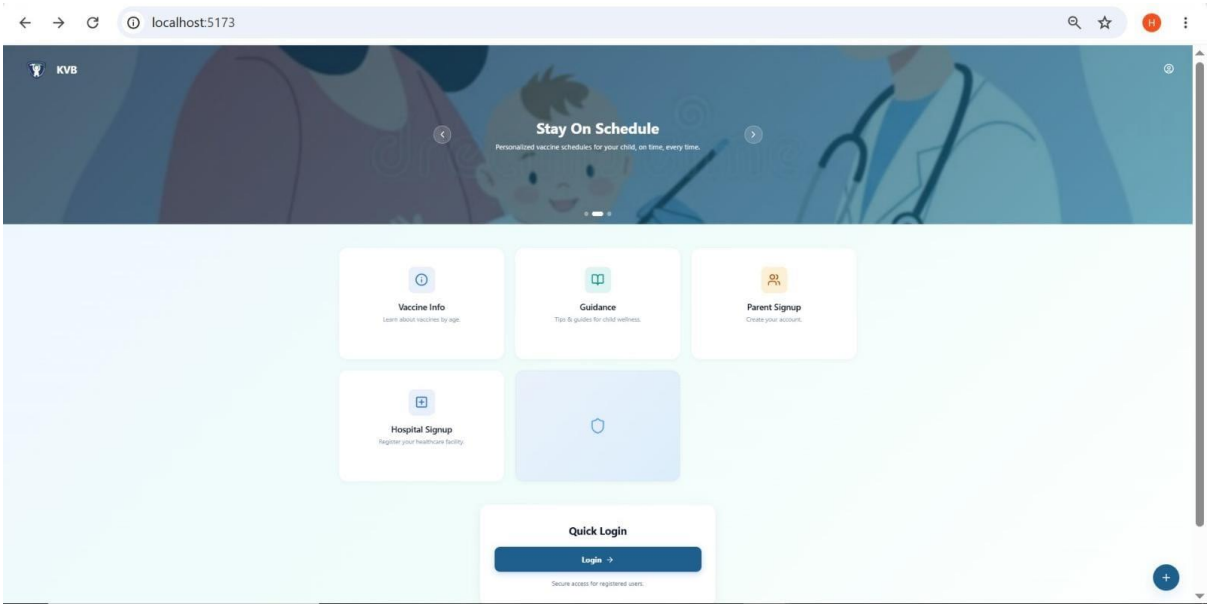
- Home Page
- Parent Signup
- Hospital Signup
- Login
- Parent Dashboard
- Child Booking Form
- Child Registration msg
- Available Slots

2. Tools Used for Prototype

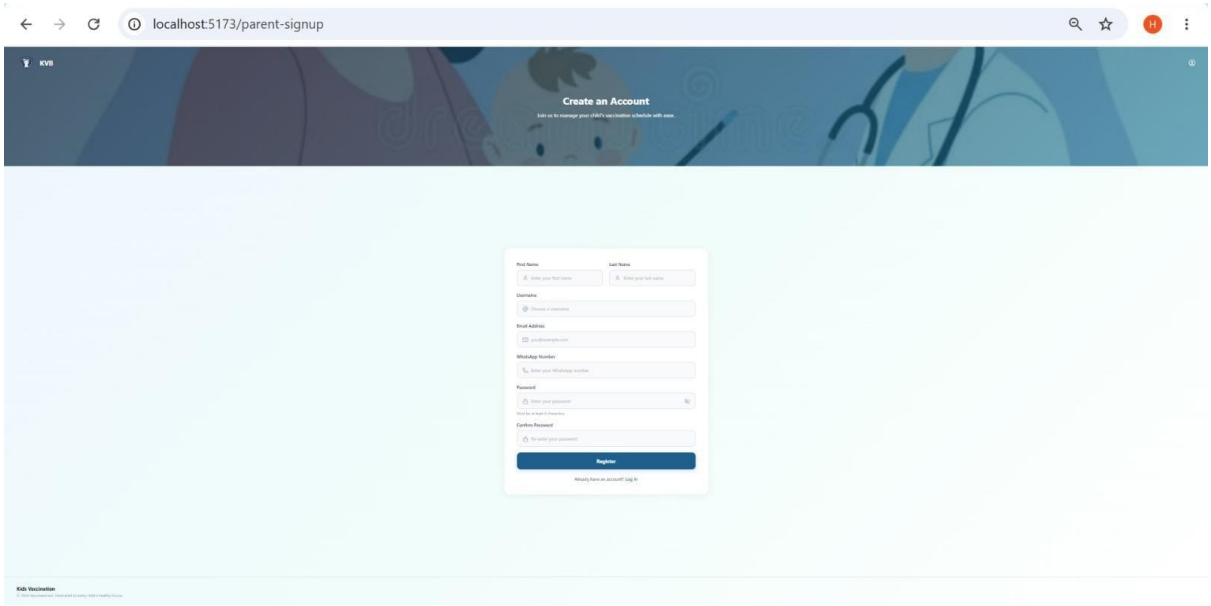
- **Frontend**: React.js
- **Backend**: Node.js with Express
- **Database**: MongoDB
- **API Testing**: browser DevTools (Network tab)
- **Version Control**: GitHub

3. Screens of key Interfaces

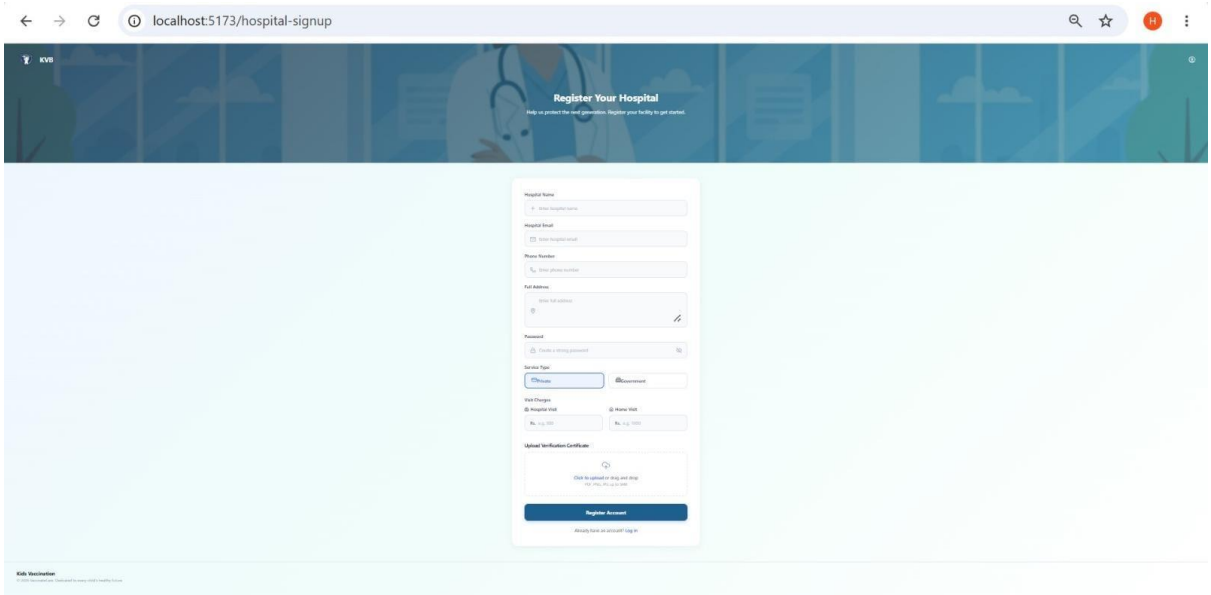
Home Page:



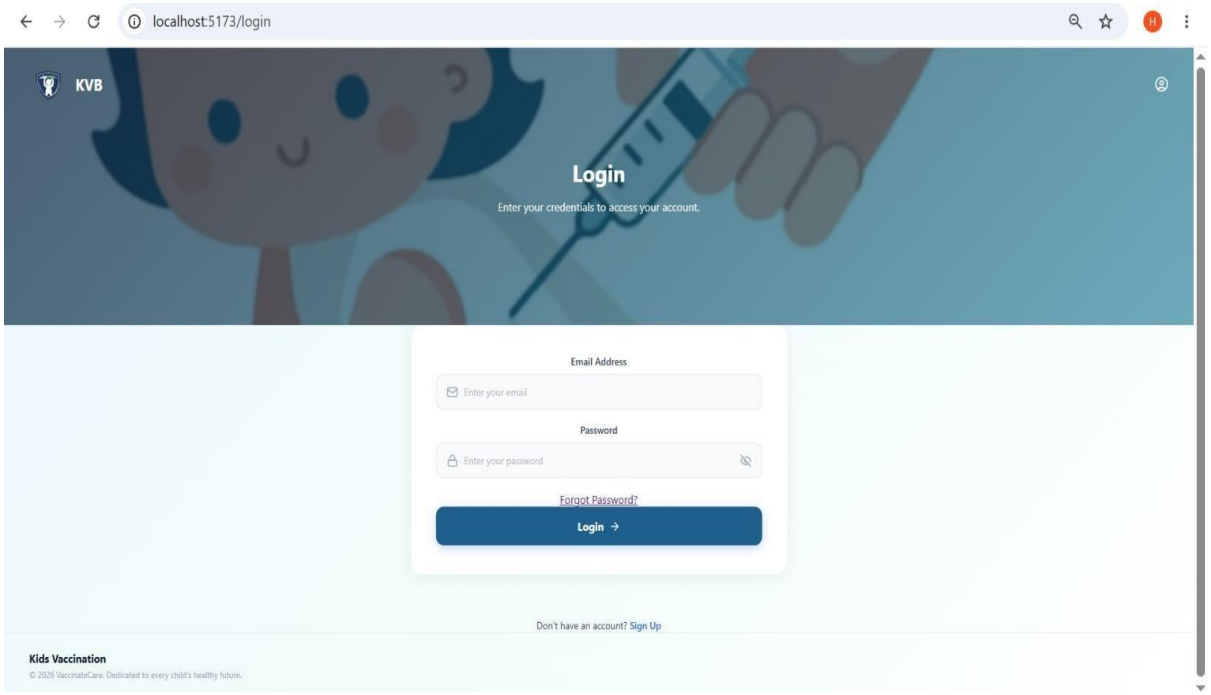
Parent Signup:



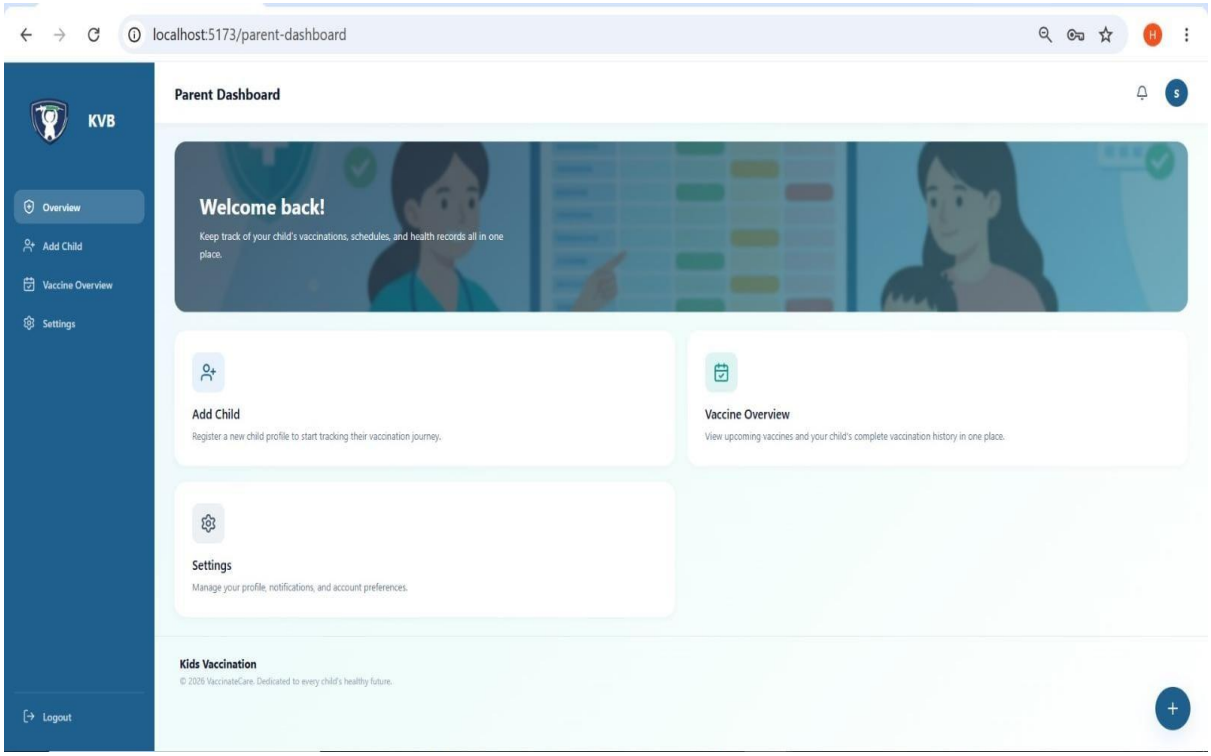
Hospital Signup:



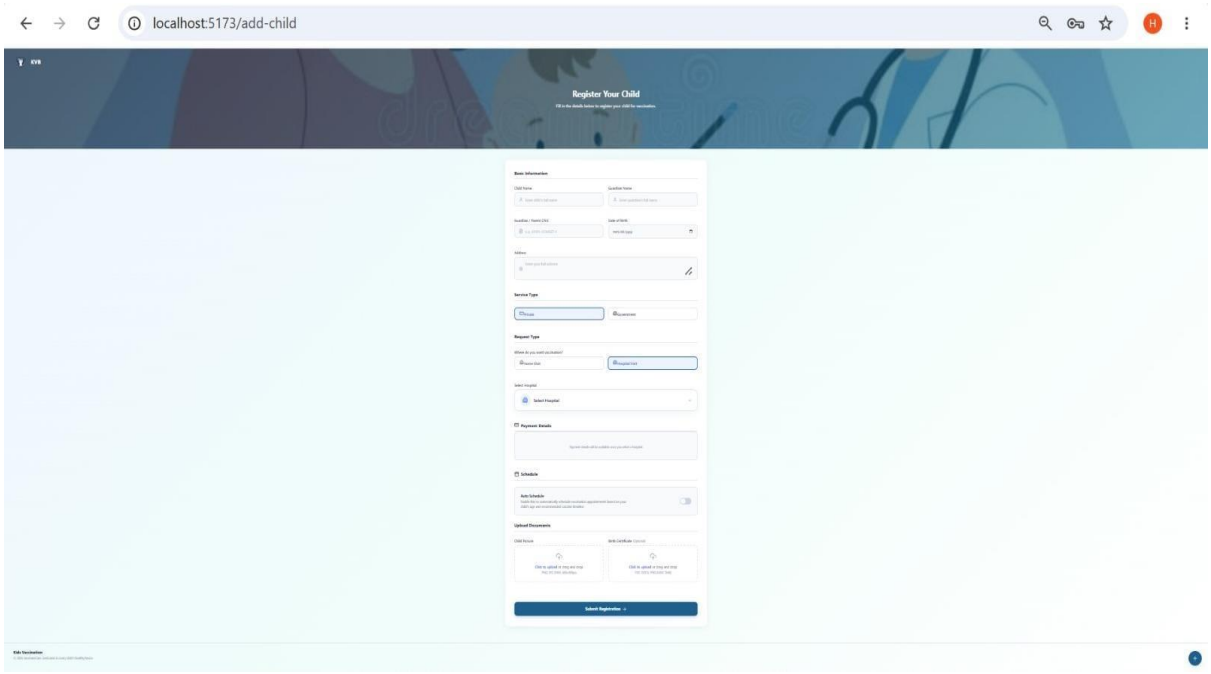
Login Page:



Parent Dashboard:



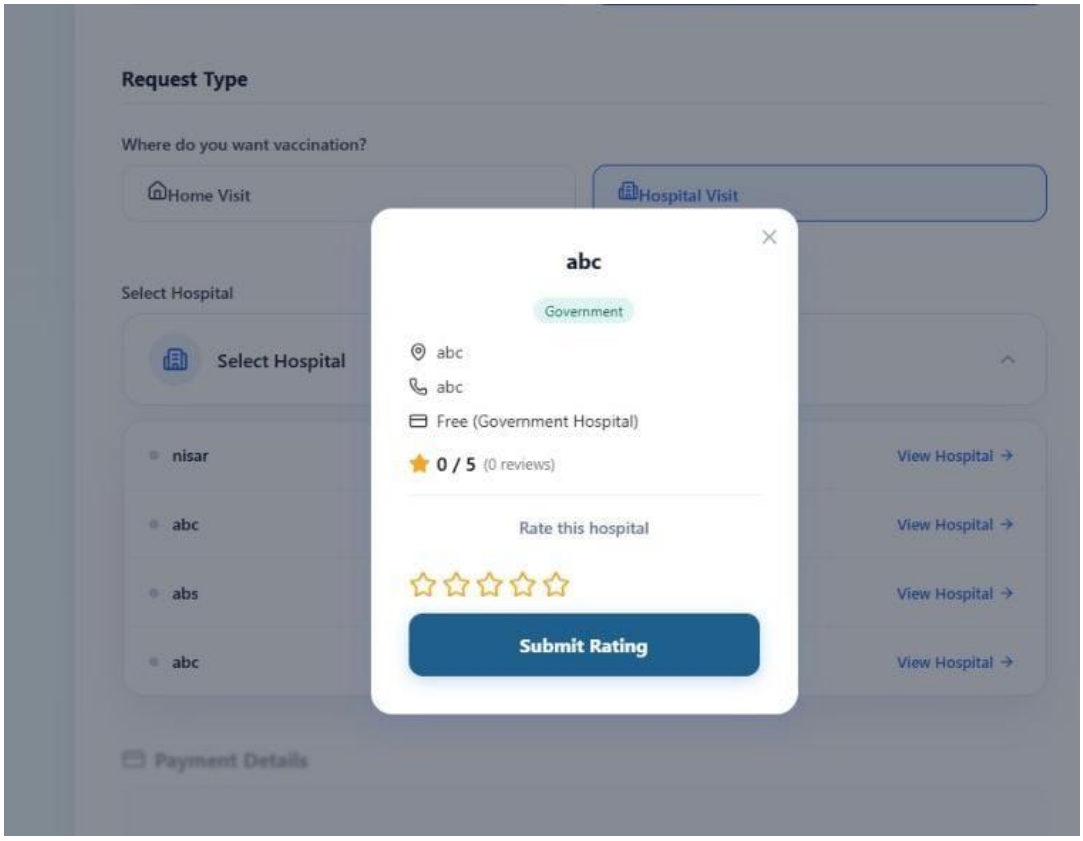
Child Booking Form:



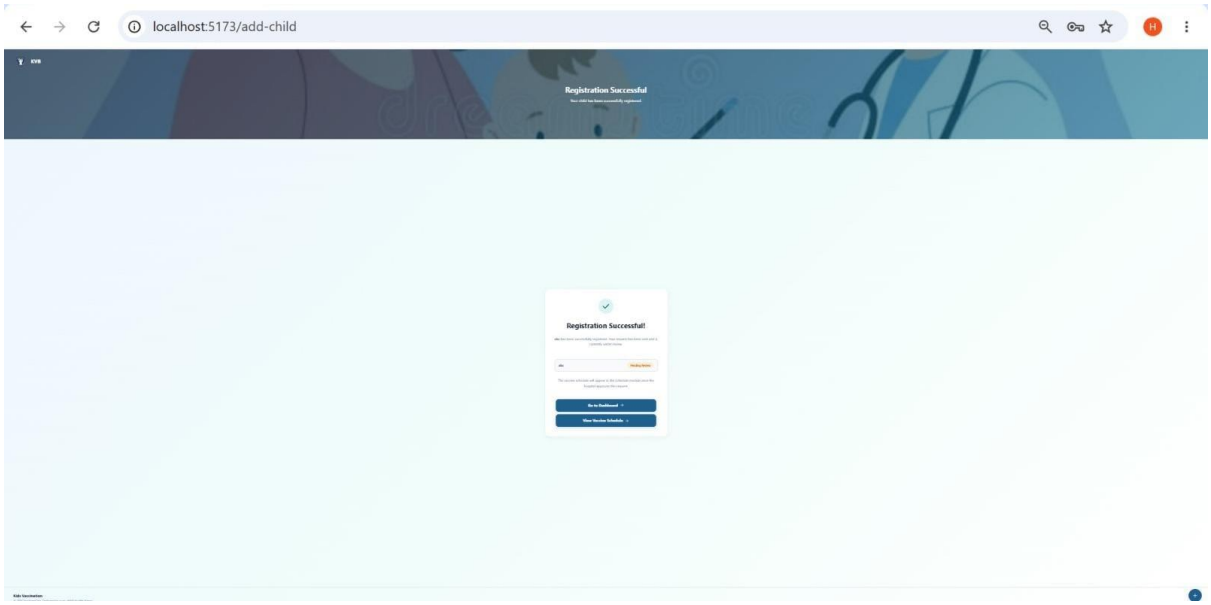
Available Slots:



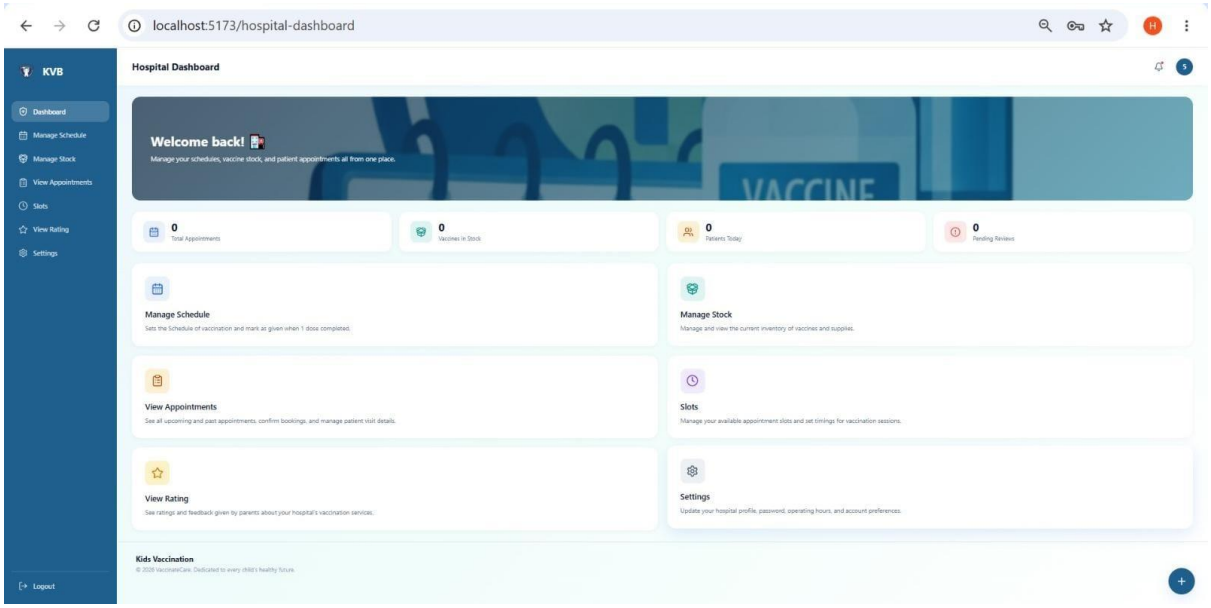
Hospital's Profile:



Child Registration msg:



Hospital Dashboard:



Admin Dashboard:

